

claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_b3a30c42-346\agent-a63df8db35a1999fb.jsonl`  
- SHA-256: `11e7de258989fd16059a5c098c2c1a1e0577a22380525aa552092f3907f4ea69`  
- Source modified: `2026-06-16T02:44:35+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_b3a30c42-346`  
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

Transcript

1. user (2026-06-16T02:36:05.176Z)

You ensure nothing important is missing from this design. Identify gaps and for each give a concrete recommendation: CI wiring (NOTE: CI is currently down on a GitHub Actions billing outage ? what is the gate meanwhile?); the exact snapshot-update developer workflow & docs; reproducibility/regeneration of fixtures if the detector improves; label-version bumps (Roora v8) and how fixtures track them; alignment with docs/QUALITY_ITERATIONS.md (ablation backbone), docs/EXPERIMENT_HARNESS.md (shadow eval), and the 'weak signals retained / default-OFF feature columns' directive; metrics-vs-base-branch comparison; handling MULTIPLE detectors (WASB/TrackNet) and the detector-agnostic candidate table; contributor onboarding docs; test runtime as fixtures grow; file-placement rules. List each missing piece with a recommendation.

DESIGN UNDER REVIEW (JSON):

```
{  
  "title": "Committed video-free golden-set regression ? unified design (frozen trajectory-CSV +  
feature substrate, dual gates, synthetic-first)",  
  "goal": "Make per-video rally-segmentation regression runnable end-to-end from a clean `git  
clone` / CI with ZERO video, GPU, WSL, DB, or network ? so a contributor on another machine  
refactoring the deterministic pipeline gets a red/green signal, and any intentional behaviour  
change shows up as a reviewable snapshot diff. Achieve this by committing a tiny, version-  
tagged, numeric-only per-video fixture corpus and TWO complementary auto-discovered pytest gates  
(a characterization/snapshot gate for refactor side-effects, and a quality-floor gate vs golden  
labels tied to the ablation track), reusing the entire shipped eval stack verbatim with  
essentially no new scoring math. This closes the concrete gap that  
`tests/test_served_gate_a_regression.py` and the golden-litmus tests SKIP in CI today because  
the corpus lives only in gitignored `output/` / private OneDrive.",  
  "buildsOnExisting": [  
    "backend/eval/metrics.py::score/match_intervals/interval_iou ? greedy, deterministic (sort  
key (-iou,pi,gi)), empty->0.0 not NaN; the ONLY scoring math, reused verbatim by both gates",  
    "backend/eval/segmentation_metrics.py::f1_at_overlaps/segment_count_ratio ? the over-seg  
signal tIoU hides, frozen in the snapshot",  
    "backend/eval/experiment.py::VideoCandidates/Variant/CONTROL/predict/evaluate_variant/compar  
e/honest_summary ? the pure LOVO scoring engine; predict()->gate->learned-filter->merge_overlaps  
is the deterministic re-score both gates call",  
    "backend/eval/fusion_compare.py::load_videos ? already globs *_golden_features.json into  
VideoCandidates; the quality-floor gate reuses it pointed at the COMMITTED dir instead of  
gitignored output/",  
    "backend/eval/windowing.py::WindowingPreset/SERVED/PROPOSAL/build_windows/windows_to_intervals ?  
the single-sourced operating point that closed the eval!=serve Gate-A trap; the fixture pins  
which preset it was frozen at",  
    "backend/eval/distill_local.py::build_candidates + FEATURE_NAMES (the 5 shuttle feats) ?  
trajectory CSV -> (intervals, shuttle feature rows) under a named preset; the exact transform  
the extractor and characterization gate call",  
    "backend/pipeline/detectors/tracknet_runner.py::parse_trajectory_csv (Frame,Visibility,X,Y)  
+ trajectory_to_action_windows ? the pure windowing 'brain' under characterization",  
    "backend/pipeline/detectors/rally_gate.py (net_crossings Gate A) + fusion_features.py
```

```
(PERSON_FEATURE_NAMES, default-OFF) ? pure per-signal feature producers",
"backend/eval/regression.py::gt_hash/save_baseline/load_baselines/regress/RegressionResult +
DEFAULT_BASELINE_PATH=eval_baselines/regression_baseline.json + the exit 0/1/3
incommensurability-guard contract (IoU/detector/gt_hash mismatch -> flagged, not silently
compared)",
"backend/eval/eval_stats.py::paired_delta_ci/mean_with_ci/sign test ? deterministic (default
seed=0); the 'NUMBERS INFORM' honest-stats the quality-floor gate emits and the snapshot
freezes",
"backend/eval/fusion_golden.py::SHUTTLE_FEATURE_NAMES/PERSON_FEATURE_NAMES/extract_video ?
the SAME extractor the owner already runs per golden video; emits the exact committed shape, so
'add a video' needs no new producer",
"tests/test_served_gate_a_regression.py ? the manifest-driven, per-video, skip-if-absent
template (env-overridable MANIFEST, _golden_specs() presence filter, _skip decorator,
Path(__file__)-anchored repo root); the new gates mirror it but RUN in CI because fixtures are
committed; its _PROPOSAL=(0.005,1.0,0.5) numbers are the proof the snapshot must record
windowing provenance",
"eval_baselines/{heuristic_lovo_n6.json, experiment_leaderboard.json,
gemini_wrapper_2026-06-10.json} ? the proven TRACKED home + provenance-carrying JSON convention
(name/description/`computed` repro cmd + label_set_hash)",
"backend/infrastructure/database.py PRAGMA user_version migration ladder (v5) + the
detector-agnostic 'common columns + stats JSON keyed by detector string' pattern ? mirrored by
the fixture schema_version + provenance design",
"run_all_tests.py + pytest.ini (testpaths=tests, no conftest, inline-tmp_path idiom) +
.github/workflows/ci.yml (ruff/mypy/import-smoke/full-suite, --cov-fail-under=70) ? the suite
the new parametrized gates slot into with no new CI job required for the core goal"
],
"chosenSeams": [
"PRIMARY FREEZE SEAM = the post-detector trajectory CSV `Frame,Visibility,X,Y` per video
(Proposal 3's choice, adopted over Proposal 1's feature-JSON-only seam). RESOLUTION OF CONFLICT:
Proposal 1 froze only the *_golden_features.json (one transform DOWNSTREAM of windowing) ?
smaller but blind to the windowing 'brain' (trajectory_to_action_windows) and Gate-A
net_crossings derivation, the exact code most likely to be refactored. The trajectory CSV is the
true GPU/WSL->pure-Python handoff; freezing here characterizes the WIDEST deterministic surface
and lets the gate re-derive features (so a feature-math regression is also caught). It is still
tiny (~20-25 B/row; a deliberately SHORT fixture clip is sub-200KB). We freeze at the CSV and
DERIVE features at test time rather than committing them as inputs.",
"REQUIRED SIDE-INPUTS frozen alongside the CSV in provenance: (fps, frame_width). NON-
NEGOTIABLE ? trajectory_to_action_windows normalizes velocity by frame_width and converts
frame->sec by fps, and _probe_fps_width's SILENT fallbacks (fps<=0->30.0, width<=0->1280.0)
would mask real regressions if a replay forgets them. All three proposals flagged this; we pin
them explicitly.",
"WINDOWING-PRESET SEAM = a named backend.eval.windowing.WindowingPreset (default SERVED,
single-sourced from IndexingConfig defaults) stamped into every fixture and snapshot. This
neutralizes the Gate-A eval!=serve cautionary tale: the snapshot records WHICH windowing it was
frozen at, so a deliberate retune trips the incommensurability guard rather than silently
passing or regressing.",
"GT SEAM = a per-video golden rally labels.csv in gt_loader conventions (start,end |
start_time,end_time | positional; decimal or MM:SS), loaded via load_gt_intervals(strict=True)
so a malformed committed label fails loudly with file:line.",
"PERSON SEAM (optional, default-OFF) = an optional per-video person_detections.json
{frames:{idx:[(tid,bbox)]}, fps, frame_width, frame_height} captured from the SAME decode as the
trajectory, so frame indices align 1:1. Synthetic tier hand-authors it for determinism; real
tier carries a codec-seek caveat in provenance.",
"EXCLUDED SEAMS (deliberately, with honest reasons): the AI/provider handoff
(provider.analyze_video ? network/non-deterministic; CONTROL emits windows verbatim / skip_ai
path, so we freeze the windows that ENTER the handoff, never provider output); the ffmpeg stitch
(VideoCompiler reel bytes are NOT reproducible across libx264/NVENC builds ? we assert the
merge_overlaps time_pairs, never bytes); DB autoincrement id + created_at (compare on
(start,end,stats) not id); the gemini and motion segmenters (no candidate stage; motion
fabricates random metadata ? only the trajectory-hybrid rally-quality family is freezable).",
],
"fixtureLayout": "NEW tracked dir `tests/fixtures/golden_regression/` (under tests/ so pytest
auto-discovers and it survives a clean checkout; NOT output/ which is gitignored ? that is
exactly why test_served_gate_a_regression skips in CI). One subdir per video keyed by a stable
slug (synthetic slug for tier-0; video_id[:12] for owner-cleared real tier-1, matching the
```

production MD5 key): `tests/fixtures/golden\_regression/<slug>/{trajectory.csv, labels.csv, expected\_features.json, expected\_segments.json, provenance.json}` (+ optional `person\_detections.json`). A single top-level repo-relative index `tests/fixtures/golden\_regression/manifest.json`: a JSON array of `{slug, trajectory, labels, features, segments, provenance, fps, frame\_width, windowing\_preset, kind:\\"synthetic\\"|\\"cleared\_real\\"}` ? the committed, repo-relative INVERSE of today's gitignored absolute-pathed output/human_lovo_manifest.json. All paths resolved via `Path(\_\_file\_\_).parent` (NEVER cwd ? agent/test cwd is unstable). A `README.md` in the dir states the minors/licensing guardrail + the freeze/refresh commands. The per-video quality-floor baseline lives at the reserved `eval\_baselines/regression\_baseline.json` (today referenced but absent ? a fresh clone hits exit-3; this design finally creates it), keyed `::final` with iou/detector/gt_hash stamps. Rationale for splitting the two homes: fixtures are TEST INPUTS (belong under tests/, CODE_MAP \$0 'a test -> tests/'); the floor baseline is a no-regression SCORE artifact (belongs in eval_baselines/, CODE_MAP \$0 'survives a clean checkout; the no-regression gate reads it'). This honors BOTH established conventions rather than overloading eval_baselines/ with raw fixtures (Proposal 1's choice) ? adopted from Proposals 2/3.",

"artifactSchema": "Every committed JSON carries a top-level `schema\_version` (int, starts at 1) + `kind`, mirroring the DB PRAGMA user_version discipline and eval_baselines' provenance convention. (1) trajectory.csv: fixed header `Frame,Visibility,X,Y`, one row/frame, ints+floats only (parse_trajectory_csv contract). (2) labels.csv: gt_loader-conformant. (3) expected_features.json: {schema_version, slug, fps, frame_width, windowing_preset:{name,velocity_thresh,merge_gap,min_window_duration}, candidates:[{start,end,shuttle:{duration,peak_velocity,mean_velocity,active_ratio,net_crossings}, person:{players_in_court,two_sided_motion,mean_player_motion,in_court_ratio,shuttle_player_colocation}, label:int}], gts:[[s,e]]} ? i.e. the EXACT fusion_golden schema (so fusion_compare.load_videos still reads it unchanged) plus only a windowing_preset stamp + schema_version. (4) expected_segments.json (the characterization snapshot): {schema_version, slug, windowing_preset, variant_spec_hash, iou_used, seed, segments:[{start,end}], num_gt, score_vs_gt:{precision,recall,f1,mean_iou,tp,fp,fn}, segmentation:{f1@10,f1@25,f1@50,segment_count_ratio}} ? the deterministic experiment.predict(CONTROL)->merge_overlaps + metrics.score output, floats stored rounded to 6 dp for byte-stable diffs. (5) provenance.json: {schema_version, slug, video_id(real only), detector, fps, frame_width, windowing:{vt,merge_gap,min_window}, gt_hash, label_set_hash, frozen_at(ISO), freeze_command, source:{license:\\"owned|ShuttleSet|synthetic\\", owned_or_synthetic, minors_free:true, consent_ref}, tool_versions:{python}}. The 'common fields + named-feature blocks + version+provenance stamp' design mirrors the DB's detector-agnostic 'columns + stats JSON keyed by detector'. The fingerprints (gt_hash, label_set_hash, Variant.spec_hash, WindowingPreset name+tuple) travel with the snapshot so any comparison against changed labels/recipe/windowing is flagged incommensurable, not silently trusted ? reusing the shipped regression.py/experiment.py guard contract rather than inventing a parallel scheme.",

"extractionTool": "A new `backend/eval/freeze\_fixture.py` CLI (thin composition over SHIPPED functions ? NO new transforms), used both to FREEZE and to REFRESH. `python -m backend.eval.freeze\_fixture --manifest <local> --slug <slug> --out tests/fixtures/golden\_regression/<slug> [--preset served] --license owned --minors-free --consent-ref <doc>`: (1) copies/normalizes trajectory CSV -> trajectory.csv and golden labels -> labels.csv (projecting the rich Rooru <video_id>.csv to the minimal gt_loader start,end shape, recording the projection in provenance); (2) calls distill_local.build_candidates(traj,fps,fw,preset) for intervals+shuttle features, rally_gate for net_crossings, and (only --with-person, video present) fusion_golden.extract_video for the person 5-col -> writes expected_features.json; (3) computes the snapshot via experiment.predict(CONTROL, VideoCandidates)->merge_overlaps at the pinned preset + metrics.score + segmentation_metrics -> expected_segments.json with variant_spec_hash + preset + iou + seed stamps; (4) emits provenance.json with regression.gt_hash(labels) + label_set_hash + the exact freeze_command + the licensing/minors assertion. CODE-ENFORCED GATE: it REFUSES to write a real fixture unless --minors-free is passed AND a --license is given (gives a code gate where the consent ledger is still only convention/BACKLOG). A `--refresh-snapshot [--all]` mode re-derives ONLY expected_features.json/expected_segments.json from the committed trajectory+labels after an INTENTIONAL behaviour change (no re-detection ? the trajectory is frozen), the characterization analogue of run_regression --update-baseline. CRITICAL: the extractor and BOTH gate tests call ONE shared helper in a new `backend/eval/golden\_fixtures.py` (load_manifest/load_fixture/replay_fixture, schema_version dispatch, Path(__file__)-relative resolution, CURRENT_SCHEMA const) so the generated artifact and the asserted artifact are computed by identical code. The extractor is itself unit-tested (synthetic in -> deterministic artifacts out).",

"characterizationGate": "`tests/test\_golden\_characterization.py` (NEW). Parametrized over manifest.json via `@pytest.mark.parametrize(\"slug\", sorted(load\_manifest()), ids=...)`. Per fixture: parse_trajectory_csv(trajectory.csv) -> windowing.build_windows(preset) -> re-derive shuttle features (distill_local.build_candidates) + net_crossings (rally_gate) into a VideoCandidates -> recompute segments via experiment.predict(CONTROL)->merge_overlaps + metrics.score; assert the recomputed expected_features.json AND expected_segments.json match the committed files. NEEDS NO LABELS for the windows/features assertions ? it catches ANY side-effect of a refactor to the windowing brain / feature math / merge, even one that does NOT move F1 (the kind of silent drift a quality-floor gate alone would miss). Comparison policy (from the determinism contract): EXACT equality on structural/integer fields (window counts, candidate count, net_crossings as counts, tp/fp/fn, segment count) since those carry the behavioral signal; pytest.approx(abs=1e-6) on floats (velocity, F1, IoU) so last-ULP cross-OS drift does not flap. A label_set_hash/spec_hash/preset guard asserts the snapshot's stamps match what is recomputed, so a fixture edited without re-freezing fails LOUDLY as incommensurable. Module skips (skipif) if the fixtures dir is empty ? but UNLIKE test_served_gate_a_regression it RUNS in CI because the synthetic tier is committed. This is the refactor-side-effect detector and the reviewer-trust affordance: a green snapshot = deterministic post-trajectory behaviour unchanged; a changed expected_*.json in the PR diff = an explicit, reviewable intentional change.",

"qualityFloorGate": "`tests/test\_golden\_quality\_floor.py` (NEW). Load the committed substrate via the fusion_compare.load_videos shape; score CONTROL (and the shuttle_only/fusion variants) vs the committed labels with metrics.score per video and experiment.compare for the honest aggregate; assert per-video AND mean F1 >= the committed floor in `eval\_baselines/regression\_baseline.json` (created via the freeze CLI + regression.save_baseline, keyed `<slug>::final` with iou/detector/gt_hash stamps so a label or operating-point change surfaces as regression.py's incommensurability flag = a test failure with a clear message, mirroring the exit-3 semantics). Floor breach uses regression.DEFAULT_TOLERANCE (0.05). The assertion MESSAGE emits eval_stats.paired_delta_ci + the exact sign test (NUMBERS INFORM, FOUNDATIONS DECIDE ? never a bare mean) and the floor is tied doc-side to QUALITY_ITERATIONS / the ablation track. STRUCTURAL GUARD: Gate-A-style structural cues are never auto-demoted on a small-n null (weak-signals-retained directive). HONEST LIMIT stated in the gate's docstring: on the SYNTHETIC tier this measures pipeline CORRECTNESS (the math floors), not field quality; the real field-quality floor still needs the owner-box run_regression against the proprietary DB OR an owner-cleared tier-1 fixture. The two gates are complementary: characterization catches behaviour-changing refactors at constant F1; quality-floor catches a true quality drop.",

"versioning": "schema_version:int on every committed JSON (starts at 1), mirroring the DB PRAGMA user_version migration-ladder. The shared loader (backend/eval/golden_fixtures.py) dispatches on schema_version with ordered `if version < N` upgrade shims that normalize an old fixture into the current in-memory shape (additive fields default safely ? same as experiment._feature_column defaulting an absent column to 0.0). A unit test asserts every committed fixture's schema_version <= CURRENT_SCHEMA (drift fails CI, exactly as the DB version assertion does). Schema changes are additive-only (new feature column, new provenance field) ? never a breaking rename ? so the characterization gate stays valid across versions; a genuinely breaking change re-freezes all fixtures in the same PR (freeze --refresh-snapshot --all) and bumps CURRENT_SCHEMA. LABEL versioning: the rich Roora guideline is at v8; provenance records the guideline version + the rich->minimal projection so a relabel under a new guideline version is a new gt_hash -> flagged incommensurable, forcing a deliberate re-baseline. CORPUS versioning: corpus-tag (e.g. 2026-06-n6, matching GOLDEN_DATA_SHARING's convention) lets an expanded corpus preserve old frozen numbers for reproducibility of prior published results (ties the paper-coauthor empirical-backbone aim). Inputs (trajectory.csv/labels.csv) are immutable per slug once committed ? a new behavioral scenario is a NEW slug dir, not an edit, so a diff to an existing slug is always either a rare reviewed input change or a snapshot re-freeze (the intentional-change signal).",

"repoSizeStrategy": "Numeric-only, comfortably git-sized, NO git-lfs (it is installed v3.5.1 but intentionally unused ? no .gitattributes; this design keeps the repo's 'derived numbers OK in git, frames/weights/DBs never' line). The trajectory CSV is the only sizeable artifact (~20-25 B/row, one row/frame); a deliberately SHORT fixture clip (synthetic ~30-90s, or real clips trimmed to a few rallies) keeps each CSV well under ~200KB; expected_*.json/provenance.json are KB-scale. A 2-3 video synthetic tier-0 corpus is <~500KB; a full owner-cleared n=6 numeric set is <~2MB. GUARD AGAINST CREEP: a unit test asserts each committed trajectory.csv row-count <= a cap (e.g. 12000 rows) so nobody commits a full 4K-match trajectory, and the freeze CLI warns past the cap. Prefer 3-5 SHORT cases over long matches ? characterization wants behavioral coverage of code branches (a quiet trajectory, a clean high-velocity rally, an occlusion gap that resets the track, a held-shuttle stretch that exercises Gate A, a multi-window video), not footage volume. output/ stays gitignored and the OneDrive

sync remains the home for the FULL proprietary corpus; the repo holds only the small committed regression subset. Fixtures are tests/-only (pyproject packages=[backend,training]) so they are not packaged into the wheel ? correct, tests aren't shipped.",

"licensingHandling": "This is the load-bearing constraint and an OWNER-GATED reversal of GOLDEN_DATA_SHARING.md (which keeps proprietary-derived per-video artifacts in private OneDrive, never committed; guardrail 'features derive from the Proprietary corpus -> keep the OneDrive folder private', 'do not symlink the shared folder into a tracked path'). RECONCILIATION (a TWO-TIER design that lands the framework WITHOUT waiting on the decision): TIER-0 SYNTHETIC fixtures (hand-authored Frame,Visibility,X,Y trajectories + synthetic labels, no real footage, no players) are licensing/privacy-clean BY CONSTRUCTION and ship with NO owner gate ? they make both gates CI-real on day one and are sufficient for CHARACTERIZATION (we lock code behaviour, not validate real-world accuracy). TIER-1 real fixtures are added ONLY on explicit owner sign-off, ONLY as DERIVED de-identified NUMERIC streams (frame index, pixel coords, scale/translation-invariant scalar features, interval labels) ? never frames/imagery, so no biometric/identifiable content; sourced ONLY from owned/licensed MINORS-FREE footage (founder collection / ShuttleSet, never end-user uploads, never broadcast scraping), each carrying provenance.json {license, owned_or_synthetic, minors_free:true, consent_ref}. The freeze CLI's --license/--minors-free refusal is the code-enforced gate. Detectors producing the streams (WASB/TrackNet) are MIT; NO paid-LLM (Gemini) output is committed as a training/data source (labels are Roora work-for-hire or synthetic). The PR introducing tier-1 must cite the owner approval and reconcile GOLDEN_DATA_SHARING.md (committed regression SUBSET vs the OneDrive full corpus). HONEST CAVEAT: even derived trajectories encode something about proprietary footage ? the synthetic-first default sidesteps this entirely and is the recommended path.",

"determinismHandling": "Freeze ONLY at pure-Python seams BEFORE every non-deterministic step. (1) FLOATS (velocity=(dist/frame_width)*(fps/dframes); rally_gate medians; normalized person motion; F1/IoU) are IEEE-754 double ops, platform-stable but asserted with pytest.approx(abs=1e-6), never exact ==; structural/integer fields (window counts, net_crossings counts, tp/fp/fn, segment counts) asserted EXACT since they carry the behavioral signal. (2) fps/frame_width are NOT in the CSV (probed from video via cv2 with SILENT 30.0/1280.0 fallbacks) -> PINNED in provenance + every feature/segment file's windowing block, so a replay never hits the fallback. (3) windowing operating point pinned to a named WindowingPreset (default SERVED, single-sourced from IndexingConfig) and stamped into expected_segments.json ? neutralizes the Gate-A eval!=serve trap (+0.074 proposal vs -0.008 served); a deliberate retune trips the incommensurability guard. (4) bootstrap CI + sign test are seeded (eval_stats default seed=0) so the honest-stats are reproducible and snapshot-able; seed is stamped in the snapshot and passed explicitly. (5) EXCLUDED entirely: provider/Gemini output + ThreadPool ordering (seam stops before the AI handoff ? CONTROL emits windows verbatim); DB autoincrement id + created_at (compare on (start,end,stats)); ffmpeg/codecs reel bytes (assert time_pairs + merge_overlaps, never bytes). (6) LINE ENDINGS: add a .gitattributes entry pinning tests/fixtures/golden_regression/** to LF (text eol=lf) so CSV/JSON are byte-identical on Windows, WSL, and Linux CI ? this is the one new (and contained) .gitattributes use and the cross-OS linchpin; parse_trajectory_csv already opens with newline='' so CRLF/LF never changes parsed rows. (7) gt_loader reads utf-8-sig (BOM-tolerant) ? keep fixtures UTF-8. RESIDUAL RISK (stated plainly): pytest.approx(abs=1e-6) is a small bounded blind spot for sub-tolerance float drift ? chosen far below any behaviorally meaningful change in seconds/F1, and integer/structural fields stay exact, so it is negligible. Person per-frame detections (cv2 frame-seek ±frames by codec) are hand-authored for the synthetic tier (fully deterministic); real person fixtures carry a codec-seek caveat in provenance and stay default-OFF.",

"ciIntegration": "CORE GOAL NEEDS ZERO NEW CI WIRING: both gates are plain pytest auto-discovered by the existing `testpaths = tests` / run_all_tests.py lane that already runs in .github/workflows/ci.yml under the 70% coverage floor, and they run in the no-GPU/no-cv2/no-network lane because the committed fixtures need none of those. KEY: import only the pure windowing/metrics/experiment path (parse_trajectory_csv, trajectory_to_action_windows, rally_gate, distill_local.build_candidates, experiment, metrics, segmentation_metrics) so the gates run even without cv2 ? preferred over pytest.importorskip('cv2') so they DO execute in CI (the whole point: test_served_gate_a_regression skips today). The synthetic tier-0 guarantees fixtures exist in CI. OPTIONAL (second PR, redundant with the pytest assertion which already gates merges): a dedicated `golden\_regression` CI job for a clean red/green signal, and/or a standalone `python -m backend.eval.freeze\_fixture --check` exit 0/1/3 CLI mirroring run_regression.py for a pre-PR local gate. SEPARATE owner-box-only workflow (out of PR CI): the existing run_regression.py against the real proprietary DB for the full corpus ? the DB path stays the owner-box gate. No custom pytest marker is registered (rely on parametrize + path-glob + skipif, matching the zero-conf test/zero-custom-marker convention) to avoid PytestUnknownMarkWarning.",

"contributorWorkflow": "CONTRIBUTOR (any machine, no video/GPU): clones, runs `python`

run_all_tests.py`. A behaviour-preserving refactor -> both gates green -> push with confidence. A failure shows exactly which slug/field changed (e.g. 'held_shuttle expected_segments f1 0.157->0.142' or 'kushagra net_crossings count changed'). They then decide: (a) unintended side-effect -> fix code until the snapshot matches; or (b) intended change -> run `python -m backend.eval.freeze\_fixture --refresh-snapshot --all` (regenerates expected_features/expected_segments from the committed trajectory+labels, NO re-detection) and, if quality moved, `run\_regression.py --update-baseline`/re-run the floor write ? BOTH land in the SAME PR as the code change, the diff reviewed. The PR description states which gate moved and why (a characterization diff is mechanical; a floor change cites the QUALITY_ITERATIONS ablation). REVIEWER: if expected_*.json is unchanged, the change is provably behaviour-preserving across the frozen seam (within documented limits); if changed, the precise before/after numbers are in the diff to judge. Rollback of a bad change = revert the code; fixtures/baseline are the source of truth, never hand-edited. ADD a video: freeze_fixture --slug <s> --license owned --minors-free --consent-ref <doc> -> writes the 5 artifacts + appends to manifest.json -> commit explicit paths. BRANCH HYGIENE (shared-checkout safety, per CLAUDE.md): git fetch origin; git checkout -b <name> origin/master; stage explicit paths only (the fixtures + tool + tests + baseline ? NEVER git add -A, and avoid the untracked artifacts/ dir + weights); re-verify HEAD before commit; one PR per phase, no stacking; do not open a PR until the owner asks. README.md in the fixtures dir documents this loop + the minors/licensing guardrail so a future contributor cannot freeze an unvetted video.",

"growthStory": "The corpus auto-grows with near-zero marginal effort because the SAME fusion_golden extractor the owner already runs per golden video emits the committed shape ? 'add a video' is freeze_fixture --slug <s> + commit, and the new video is instantly a CI-enforced regression case (both gates pick it up via the manifest glob). As the Roora golden corpus grows (n=6 -> larger), each new owner-cleared video is one more parametrized case; the honest-stats (bootstrap CI + sign test, seed=0) tighten as n rises, so the quality-floor gate's verdicts get more trustworthy automatically. NEW CUES extend without regression: a future cue (audio, shuttle-drop) is just a new named feature block in expected_features.json + a Variant feature_name ? the same gates score it default-OFF with zero regression (the weak-signals-retained directive: non-significant cues stay retained-as-columns, never deleted on a small-n null). NEW WINDOWING/operating points are new presets stamped into new fixtures, preserving old frozen numbers under their corpus-tag (reproducibility of prior published results ? the paper-coauthor backbone). NEW DETECTORS reuse the same trajectory-CSV seam (the schema is detector-agnostic via the detector string in provenance). The schema_version ladder absorbs additive growth so old fixtures keep loading. Net: the framework is built once on synthetic data, then scales by accretion ? every golden video the vendor pipeline produces becomes a permanent, video-free, CI-runnable regression anchor.",

"rolloutPhases": [
 {
 "phase": "PR-1: Framework + synthetic tier-0 (NO owner gate ? ships immediately)",
 "deliverable": "tests/fixtures/golden_regression/ + README.md (guardrails + freeze/refresh loop); backend/eval/golden_fixtures.py (load_manifest/load_fixture/replay_fixture, schema_version dispatch, Path(__file__)-relative, CURRENT_SCHEMA=1); a .gitattributes LF pin for the fixtures dir; 3-5 hand-authored SYNTHETIC tier-0 fixtures (clean rally / held-shuttle Gate-A cut / occlusion break / quiet trajectory / multi-window), tool-generated expected_*.json + provenance.json; manifest.json.",
 "prScope": "One PR branched from origin/master. Establishes the committed-fixture convention + the shared loader on licensing-clean synthetic data. No scoring math added. Stage explicit paths only."
 },
 {
 "phase": "PR-2: The freeze/refresh extractor + its unit tests",
 "deliverable": "backend/eval/freeze_fixture.py composing distill_local.build_candidates + rally_gate + experiment.predict(CONTROL)+merge_overlaps + regression.gt_hash; --license/--minors-free refusal gate; --refresh-snapshot[--all]; --check exit 0/1/3 (optional). Unit test: synthetic in -> deterministic artifacts out; re-running --refresh-snapshot is idempotent.",
 "prScope": "One PR. The tool is the single source of truth for both generation and the gates' replay (shared helper). Re-generates PR-1's synthetic fixtures so they are tool-produced, not hand-typed."
 },
 {
 "phase": "PR-3: Characterization gate",
 "deliverable": "tests/test_golden_characterization.py ? parametrized over manifest.json, recompute features+segments from the frozen trajectory under the pinned preset, assert exact-on-

```

structural + approx(1e-6)-on-float match vs expected_features.json/expected_segments.json;
incommensurability/preset/hash guard; assert no DB-id/timestamp/byte fields touched. Plus the
schema_version-drift test and the trajectory row-count cap test.",
  "prScope": "One PR. The refactor-side-effect detector. Runs in CI on the synthetic tier;
confirm green + coverage >=70% locally."
},
{
  "phase": "PR-4: Quality-floor gate + the first committed regression_baseline.json",
  "deliverable": "tests/test_golden_quality_floor.py ? load substrate via
fusion_compare.load_videos shape, score CONTROL/shuttle_only/fusion vs labels, assert per-
video+mean F1 >= committed floor; emit eval_stats CI + sign test in the failure message. Create
eval_baselines/regression_baseline.json (finally fills the reserved-but-absent path) keyed
<slug>::final via regression.save_baseline with iou/detector/gt_hash stamps.",
  "prScope": "One PR. Ties the floor to QUALITY_ITERATIONS/ablation. Synthetic-tier floors
measure correctness; documented as such."
},
{
  "phase": "PR-5 (DOCS): document the framework + update CODE_MAP",
  "deliverable": "New docs/GOLDEN_REGRESSION_FIXTURES.md (or a section in EVALUATION.md)
covering what the gates DO and do NOT guarantee (false-safety limits), the freeze/refresh
workflow, the reviewer read-the-diff responsibility, the two-tier licensing reconciliation;
cross-link from EXPERIMENT_HARNESS.md/GOLDEN_DATA_SHARING.md; add the
tests/fixtures/golden_regression/ row to CODE_MAP $0 file-placement table.",
  "prScope": "One docs PR (or folded into PR-1/PR-3). Keeps live docs honest."
},
{
  "phase": "PR-6 (OWNER-GATED, separate): tier-1 owner-cleared real fixtures",
  "deliverable": "On explicit owner sign-off: add owner-cleared, minors-free, de-identified
real NUMERIC fixtures via freeze_fixture (citing the approval), and reconcile
GOLDEN_DATA_SHARING.md (committed regression subset vs OneDrive full corpus). Lights up the
real-footage edge cases (occlusion, codec jitter) the synthetic tier cannot.",
  "prScope": "One PR, owner-approved. Purely additive to the framework already shipped in
PR-1..5. Do not land unilaterally ? reverses a product/privacy convention."
}
],
"recommendedDocHome": "Create a NEW doc `docs/GOLDEN_REGRESSION_FIXTURES.md` as the canonical
home (the framework is distinct enough ? committed video-free fixtures + dual gates + freeze
tool ? that bolting it into EVALUATION.md would bury it). It must: (1) state plainly what the
gates DO and do NOT guarantee (the characterization false-safety limit ? green = deterministic
POST-trajectory behaviour unchanged for the frozen cases, NOT 'correct' and NOT covering the
detector/video-IO/provider/stitch); (2) document the freeze/refresh contributor loop and the
reviewer's read-the-diff responsibility; (3) record the two-tier licensing reconciliation with
GOLDEN_DATA_SHARING.md. CROSS-LINK it from: docs/EVALUATION.md (the scoring mechanics),
docs/EXPERIMENT_HARNESS.md (the Variant/compare engine it reuses + the P3 CI-promotion-gate it
complements), docs/GOLDEN_DATA_SHARING.md (the private-OneDrive convention it partially reverses
for the committed subset), and QUALITY_ITERATIONS.md (the ablation the quality-floor ties to).
ALSO add the `tests/fixtures/golden_regression/` row to the CODE_MAP $0 file-placement table and
a TEST MAP entry. Update docs/README.md doc index + the memory current-state/session-handoff at
the PR-1 checkpoint.",
"openOwnerQuestions": [
  "PRIMARY (product/privacy, gates tier-1): Approve reversing GOLDEN_DATA_SHARING.md for a
TINY committed regression SUBSET of DERIVED, de-identified, minors-free, owned-footage NUMERIC
streams (trajectory rows + interval labels + scalar features ? never frames)? The framework
(PR-1..5) ships on synthetic data without this; only tier-1 real fixtures need sign-off.",
  "Which n of the owned n=6 golden videos (if any) are cleared as minors-free + license-clean
for a committed numeric subset, and is ShuttleSet's license compatible with committing DERIVED
trajectory/feature numbers to a (public?) repo?",
  "Is this repo public or private? If public, even derived numeric streams from proprietary
footage are a stronger disclosure than if private ? changes the tier-1 risk calculus and
possibly the synthetic-only decision.",
  "Should the committed quality-floor numbers (eval_baselines/regression_baseline.json) be
tied to the SERVED windowing preset only, or also track the PROPOSAL preset (where Gate-A's
+0.074 lives)? Recommendation: freeze SERVED as the floor (the operating point users get),
record PROPOSAL as a documented control ? confirm.",
  "Confirm the floor tolerance (regression.DEFAULT_TOLERANCE=0.05) and the trajectory row-

```

count cap (~12000) are acceptable, or set preferred values.",

"Should tier-1 real fixtures key by video_id[:12] (the production MD5 key, so a fixture maps to a real DB row) or by an opaque de-identification slug (stronger privacy)? Trade-off: traceability vs disclosure."

```
],
"onestLimits": [
"CHARACTERIZATION = behaviour-locking, NOT correctness. A green snapshot proves only that
DETERMINISTIC POST-TRAJECTORY behaviour is unchanged for the FROZEN cases; it encodes CURRENT
behaviour including any current bugs. 'Passing' means 'behaviour unchanged', never 'behaviour
correct'. Easy to misread as a quality gate ? the docs must say this loudly.",
"The gates do NOT cover: the WASB/TrackNet detector itself (needs video/GPU), video-
IO/decode (_probe_fps_width, frame seeking, ffmpeg), the AI/provider handoff (excluded/stubbed),
reel-byte output (excluded), or any code path the fixtures do not exercise. A refactor that
breaks decoding or the detector passes these gates green. The owner-box run_regression against
the real DB remains the only check of the full chain.",
"Coverage is only as good as the fixture cases ? an untested windowing/Gate-A branch has
zero protection; adding a case when a new code path appears is a discipline requirement, not
enforced.",
"--refresh-snapshot can be abused: a contributor can rubber-stamp a real regression by
regenerating the snapshot without justification. The mitigation is social/review (the diff is
visible and must be explained), not technical ? the gate's strength depends on the reviewer
actually reading snapshot diffs.",
"Synthetic tier-0 exercises the PLUMBING faithfully but will NOT reproduce real-footage
failure modes (occlusion breaks, codec frame-seek jitter, real velocity distributions); tier-0
quality-floor numbers are about pipeline correctness, not field quality. Real field quality
needs owner-cleared tier-1 or the owner-box DB run.",
"Two sources of truth still coexist: the DB-backed run_regression (owner-box, full chain)
and these file-backed gates (CI, post-trajectory only). This design picks the trajectory/feature
substrate and records its provenance, but a contributor must understand the DB path is a
separate, owner-box-only gate ? they are not interchangeable.",
"splits.assert_disjoint is NOT yet wired into this path; if a committed fixture ever both
trains a pinned scorer AND serves as a regression baseline, train/eval leakage is possible until
that guard is connected (recommend wiring it when/if a learned variant is pinned against these
fixtures).",
"pytest.approx(abs=1e-6) is a small bounded blind spot for sub-tolerance float drift
(mitigated by exact-on-structural-fields). And the dev container here cannot run the full
pipeline (no GPU/torch/cv2/numpy) ? the synthetic-tier gates are verifiable here, but tier-1
extraction and the owner-box DB regression were NOT verifiable in this environment."
]
}
```

CONTEXT (JSON):

```
[
{
"area": "Existing regression/golden/eval infrastructure in backend/eval (badminton rally
detector) ? for a \"video-free regression from committed structured files\" design",
"summary": "The repo already has a mature, layered, mostly-pure eval stack under
backend/eval/ that is video-free at the SCORING boundary: pure interval metrics (metrics.py,
segmentation_metrics.py), one canonical GT CSV loader (gt_loader.py), a DB-read harness
(harness.py), a baseline-snapshot regression gate (regression.py + run_regression.py, exit
0/1/3) and a sibling eval harness CLI (run_eval.py), an A/B experiment engine (experiment.py)
with a committed leaderboard, an ablation driver (ablation.py), and two offline replay-substrate
producers (fusion_golden.py -> per-video *_golden_features.json; export_wasb_segments.py).
HOWEVER, none of the existing flows runs from committed-in-repo structured files end-to-end. Two
distinct \"sources of truth\" exist: (a) the DB path (harness/regression read predictions from a
SQLite DB that only the GPU pipeline can populate), and (b) the trajectory/feature path
(calibrate_wasb, distill_local, fusion_golden, served_gate_a, ablation read WASB trajectory CSVs
/ golden_features JSON / .rallies.csv whose paths come from output/human_lovo_manifest.json ?
which is gitignored and points at absolute paths OUTSIDE the repo). ablation.py is the closest
existing thing to a video-free, file-driven regression: it loads *_golden_features.json from a
directory and runs entirely on CPU with no DB and no video ? but those JSONs are not committed.
The committed structured artifacts that DO survive a clean checkout live in eval_baselines/
(tracked): heuristic_lovo_n6.json (the LOVO floor), experiment_leaderboard.json,
gemini_wrapper_2026-06-10.json. A design can reuse the entire scoring/loader/stats/gate stack
as-is and only needs to ADD committed golden fixtures (labels + a frozen predictions/feature
```


substrate) plus a thin file-fed runner.",

"keyFindings": [

"(1) WHAT EXISTS / HOW RUN: Two CLI entrypoints at repo root. run_eval.py (-> backend.eval.harness.evaluate) scores DB-stored predictions vs a GT CSV and prints/JSON-dumps P/R/F1 per stage; read-only, no gate. run_regression.py (-> backend.eval.regression.regress_with_provider) scores DB predictions vs GT from an annotation provider tier and compares to a snapshot of baseline JSON, returning exit 0=ok / 1=regressed / 3=no-baseline-to-compare. Both read predictions from SQLite (harness.get_predictions: 'candidates'->db.query_candidate_segments, 'final'->db.query_play_segments). backend.eval.experiment.compare/evaluate_variant is the A/B engine; backend.eval.ablation.run_ablation is the LOSO/LOGO driver (CLI: python -m backend.eval.ablation --features-dir output). fusion_golden, export_wasb_segments, served_gate_a, calibrate_wasb, distill_local are all `python -m backend.eval.<mod>` CLIs. CI (.github/workflows/ci.yml) runs ONLY ruff + mypy + import-smoke + the full pytest suite (--cov-fail-under=70); it does NOT invoke run_regression/run_eval as a gate. So regression today is pytest-only (test_eval_regression.py) using inline tmp_path CSVs + in-memory Database, NEVER a committed golden run.",

"(2) ALREADY VIDEO-FREE? Partially, and in two senses. The SCORING math (metrics.py, segmentation_metrics.py) and the GT loader (gt_loader.py) are pure stdlib, no video, no GPU, no numpy. harness/regression are video-free in that they read predictions from the DB (no re-detect) ? but the DB itself can only be populated by running the GPU pipeline on video, so 'no committed file -> no run'. The CLOSEST existing video-free-from-files flow is ablation.py: load_videos(features_dir) reads every *_golden_features.json (intervals + shuttle/person/audio feature columns + golden gts baked in) and runs the full ablation 100% on CPU with NO DB and NO video. experiment.compare likewise runs purely on VideoCandidates objects (intervals+features+gts). So the ENGINE to do 'video-free regression from structured files' already exists end-to-end ? what is missing is COMMITTED inputs: the *_golden_features.json files are written to gitignored output/ and are not in the repo.",

"(3) GOLDEN FIXTURES: eval_baselines/ IS tracked (committed) ? NOT gitignored. It holds heuristic_lovo_n6.json (mean_f1 0.480 + per-video F1 floor for n=6 corpus), experiment_leaderboard.json (append-only A/B records with honest ?F1 CI + sign test), gemini_wrapper_2026-06-10.json. There is NO committed regression_baseline.json (regression.DEFAULT_BASELINE_PATH='eval_baselines/regression_baseline.json' but the file does not yet exist ? it would be committed there once created via --update-baseline). There are NO committed golden label CSVs (.rallies.csv), NO committed WASB trajectory CSVs, and NO committed *_golden_features.json. The golden corpus lives entirely OUTSIDE the repo: output/human_lovo_manifest.json (gitignored) lists 6 videos by name with absolute Windows paths like C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/*.csv and ../Golden Labelled/*.rallies.csv. Tests that need the golden corpus (test_served_gate_a_regression.py) skipif the manifest/files are absent (CI skips; owner box runs).",

"(4) ARTIFACT FORMATS ARE ALREADY STANDARDISED for a file-fed design: gt_loader.load_gt_intervals accepts BOTH golden CSV conventions through one reader (header start/end OR start_time/end_time, else positional col 0,1; MM:SS/HH:MM:SS or decimal seconds; strict raises with file:line, lenient logs skips). export_wasb_segments writes the golden/slicer schema CSV (rally_number,start_time,end_time,duration,start_mmss,end_mmss) plus a *_vs_golden comparison CSV (type,ref,start_time,end_time,mmss,status,iou,pred_start,pred_end,start_err_s,end_err_s). fusion_golden writes one JSON per video {name,fps,frame_width,candidates:[{start,end,shuttle{5 cols},person{5 cols},label}],gts:[{s,e}...]} consumed by ablation.load_videos and re-scorable forever with no GPU. The regression baseline JSON schema is {'<video_id>::<stage>': {video_id,stage,precision,recall,f1,iou_threshold,detector,gt_hash}} with incommensurability guards (IoU/detector/gt_hash mismatch -> flagged, not silently compared).",

"(5) FOR A 'VIDEO-FREE REGRESSION FROM COMMITTED STRUCTURED FILES' DESIGN ? REUSE vs ADD. REUSE (do not reinvent): metrics.score/match_intervals/pr_curve; segmentation_metrics.f1_at_overlaps/mean_average_precision/segment_count_ratio; gt_loader.load_gt_intervals (the one CSV reader); regression.regress/save_baseline/load_baselines/gt_hash + the exit 0/1/3 gate contract in run_regression.py; experiment.VideoCandidates + evaluate_variant + compare + record_experiment + the eval_baselines/ committed-location convention; ablation.load_videos (already reads committed-shaped golden_features JSON, DB-free, GPU-free); eval_stats CI+sign-test, splits.assert_disjoint train/eval guard. ADD: (a) commit a small golden fixture set in-repo (label CSVs in the golden schema + a FROZEN predictions/feature substrate ? either tiny *_golden_features.json or a committed predictions JSON/CSV per video) under a tracked dir (tests/ has no fixtures today; eval_baselines/ or a new tests/golden/ are the natural homes); (b) a thin file-fed predictions loader that feeds metrics/regression WITHOUT the DB (today

regression.regress only takes predictions via harness.get_predictions -> DB; you need a variant that reads predictions from a committed file); (c) commit the manifest with repo-relative paths (current output/human_lovo_manifest.json is gitignored + absolute-pathed); (d) optionally wire run_regression into CI (today it is not a CI gate)."

```

],
"dataStructures": [
  {
    "name": "RegressionResult / regression_baseline.json",
    "where": "backend/eval/regression.py
(DEFAULT_BASELINE_PATH=eval_baselines/regression_baseline.json ? file NOT yet committed)",
    "shapeOrFormat": "Baseline store JSON: { '<video_id>::<stage>':
{video_id,stage,precision,recall,f1,iou_threshold,detector,gt_hash} }. RegressionResult
dataclass adds regressed:bool, reasons:[str], has_baseline:bool, tolerance,
baseline_precision/recall. gt_hash = sha1 of sorted rounded GT intervals (detects label drift).
Incommensurability guards flag IoU/detector/gt_hash mismatch."
  },
  {
    "name": "Golden / slicer CSV schema",
    "where": "backend/eval/export_wasb_segments.py (SEG_FIELDS); read by
gt_loader.load_gt_intervals and tools.rally_slicer.load_rally_labels",
    "shapeOrFormat": "CSV columns:
rally_number,start_time,end_time,duration,start_mmss,end_mmss. The canonical GT reader
(gt_loader) also accepts the collector export form (start,end), positional headerless, and
MM:SS/HH:MM:SS or decimal-seconds timestamps. Comparison CSV (COMP_FIELDS):
type,ref,start_time,end_time,mmss,status,iou,pred_start,pred_end,start_err_s,end_err_s."
  },
  {
    "name": "*_golden_features.json (replay substrate)",
    "where": "produced by backend/eval/fusion_golden.py (run -> out_dir, default gitignored
output/); consumed by backend/eval/ablation.py load_videos and the experiment engine",
    "shapeOrFormat": "{name, fps, frame_width, candidates:[{start, end,
shuttle:{duration,peak_velocity,mean_velocity,active_ratio,net_crossings}, person:{players_in_co
urt,two_sided_motion,mean_player_motion,in_court_ratio,shuttle_player_colocation},
audio?:{audio_hit_count,audio_hit_rate,audio_hit_strength_mean}, label:int]],
gts:[{start,end},...]]. This is the GPU-free, video-free re-scoring input ? exactly the artifact
a file-driven regression would commit."
  },
  {
    "name": "human_lovo_manifest.json",
    "where": "output/human_lovo_manifest.json ? GITIGNORED (output/ is in .gitignore line
39); read by fusion_golden, served_gate_a, distill_local, calibrate_wasb CLIs",
    "shapeOrFormat": "JSON array of [{name, trajectory:<abs path to WASB CSV>, fps,
frame_width, labels:<abs path to .rallies.csv>}]. Paths are absolute and outside the repo
(C:/Users/avidu/Projects/Annotation Setup/...), so this corpus does NOT survive a clean
checkout. n=6 owned golden videos."
  },
  {
    "name": "VideoCandidates / Variant / experiment record",
    "where": "backend/eval/experiment.py; leaderboard committed at
eval_baselines/experiment_leaderboard.json",
    "shapeOrFormat": "VideoCandidates(name, intervals:[(s,e)], features:{col->[per-candidate
floats]}, gts:[(s,e)] or None). Variant = declarative re-scoring recipe
(min_crossings/min_players gates, feature_names, classifier_model, threshold, merge_gap; control
defaults are no-op). Leaderboard row: {timestamp,iou,label_set_hash,num_videos,variant_a/b:{name
,spec_hash,aggregate},delta,honest_delta_f1:{mean_delta,ci_low,ci_high,sign_test:{wins,losses,p_
value}}}}."
  },
  {
    "name": "WASB trajectory CSV",
    "where": "parsed by TrackNetRunner.parse_trajectory_csv; consumed by
calibrate_wasb.make_predict_fn, export_wasb_segments, served_gate_a,
distill_local.build_candidates",
    "shapeOrFormat": "CSV columns: Frame,Visibility,X,Y (one row per frame). The per-video
raw substrate from which candidate windows + shuttle features are derived; lives outside the
repo per the gitignored manifest."
  }
]

```

```

    },
    {
        "name": "split manifest (train/eval guard)",
        "where": "backend/eval/splits.py load_split/assert_disjoint",
        "shapeOrFormat": "JSON list of entries each with match_id (or id fallback) + split in
{'train','eval'}; raises on overlap. Pure-logic guard, not yet wired into
distill_local/run_regression (wiring deferred per its docstring).\"
    }
],
    "videoDependency": "SCORING and the existing experiment/ablation engines are fully video-
free. metrics.py, segmentation_metrics.py, gt_loader.py, splits.py, regression.py (the pure
parts) need no video/GPU/numpy. experiment.compare and ablation.run_ablation run end-to-end on
CPU from in-memory VideoCandidates / committed-shaped *_golden_features.json ? no video, no DB,
no Gemini. The DB-read path (harness.get_predictions, regression.regress_with_provider,
run_eval/run_regression) is video-free at run time BUT depends on a SQLite DB that only the GPU
pipeline (run on real video) can populate ? so without a committed file it cannot run in CI. The
FEATURE-PRODUCING step (fusion_golden person detection over the original MP4) and the served-
path windowing (served_gate_a reads cached trajectory CSVs, GPU-free with person cue OFF) need
the out-of-repo golden corpus (trajectories + .rallies.csv via the gitignored manifest). Net:
the design's goal is achievable with ZERO video by committing a frozen predictions/feature
substrate + labels and feeding the already-pure scoring/regression layer.",
    "builtVsPlanned": "BUILT & shipped (with passing pytest): pure metrics + segmentation
metrics; canonical GT loader (gt_loader, closes the two-loader fork, ADR-008 prereq); DB-read
harness + run_eval CLI; baseline-snapshot regression gate + run_regression CLI with exit 0/1/3
and incommensurability guards; experiment A/B engine
(compare/evaluate_variant/compare_unlabeled/record_experiment) with committed leaderboard;
LOSO/LOGO ablation driver with honest CI+sign-test verdicts
(earns_keep/suggestive/inert/structural_protected) + PDF export; fusion_golden feature extractor
(with run-telemetry black box); export_wasb_segments for eyeball verification; served_gate_a
production-path litmus; calibrate_wasb + distill_local; splits train/eval guard (pure-logic).
Committed fixtures: eval_baselines/{heuristic_lovo_n6.json, experiment_leaderboard.json,
gemini_wrapper_2026-06-10.json}. PLANNED / NOT built: a committed regression_baseline.json
(default path reserved, file absent); committed golden label CSVs / trajectories /
*_golden_features.json (all currently land in gitignored output/ or outside the repo); splits
wiring into distill_local/run_regression (explicitly deferred in its docstring); regression as a
CI gate (CI runs only lint/mypy/import-smoke/pytest+coverage). No 'video-free regression from
committed structured files' runner exists yet ? the engine exists, the committed inputs and the
file-fed predictions loader do not.",
    "constraints": [
        "Commercial-clean licensing: every dependency code/data/weights MIT/Apache-2.0/BSD only;
paid LLMs are inference/serving/fallback only, never a training data source.",
        "Dev container lacks GPU/torch/cv2/numpy ? the full detect pipeline cannot run here; pure-
logic eval (metrics, gt_loader, regression core, experiment, ablation, splits) DOES run here. A
committed file-driven regression must stay in the pure-logic lane to be CI-runnable.",
        "output/ is gitignored (.gitignore line 39) and *.db is gitignored (lines 36-38) ? so the
runtime DB, the manifest, golden_features JSON, and the default regression_baseline.json
location do NOT survive a clean checkout unless explicitly relocated. eval_baselines/ is the
established TRACKED home for committed eval artifacts.",
        "Honest small-n contract (owner directive 'NUMBERS INFORM, FOUNDATIONS DECIDE'): never a
bare mean ? every signal/regression delta ships a bootstrap CI + exact paired sign test
(eval_stats); weak/non-significant cues are retained default-OFF, never deleted on a small-n
null; structural guards (Gate A) are never auto-demoted on 'no lift'.",
        "Train/eval separation: split unit = match id (never clip/MD5); splits.assert_disjoint
must guard any fixture used both for training a scorer and for regression baselines.",
        "gt_hash / label_set_hash / spec_hash fingerprints must travel with any committed baseline
so a comparison against changed labels or a changed recipe is flagged incommensurable rather
than silently trusted (the existing regression.py and experiment.py guard contract).",
        "eval must not drift from serve: distill_local.PROPOSAL vs SERVED windowing are single-
sourced in backend.eval.windowing; served_gate_a pins production operating point (vel 0.02,
merge 2.0, min 2.0). A file-driven golden must record which windowing/operating-point it was
frozen at."
    ],
    "relevantPaths": [
        "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/metrics.py",
        "C:/Users/avidu/Projects/badminton-highlight-

```

```

indexer/backend/eval/segmentation_metrics.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/gt_loader.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/regression.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/harness.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/experiment.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/ablation.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/fusion_golden.py",
    "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/eval/export_wasb_segments.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/served_gate_a.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/distill_local.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/splits.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/run_eval.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/run_regression.py",
    "C:/Users/avidu/Projects/badminton-highlight-
indexer/eval_baselines/heuristic_lovo_n6.json",
    "C:/Users/avidu/Projects/badminton-highlight-
indexer/eval_baselines/experiment_leaderboard.json",
    "C:/Users/avidu/Projects/badminton-highlight-
indexer/eval_baselines/gemini_wrapper_2026-06-10.json",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/output/human_lovo_manifest.json",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_eval_regression.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_gt_loader.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_eval_harness.py",
    "C:/Users/avidu/Projects/badminton-highlight-
indexer/tests/test_fusion_golden_telemetry.py",
    "C:/Users/avidu/Projects/badminton-highlight-
indexer/tests/test_served_gate_a_regression.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_export_wasb_segments.py",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/.github/workflows/ci.yml",
    "C:/Users/avidu/Projects/badminton-highlight-indexer/.gitignore"
],
"risksGaps": [
    "No committed golden fixture survives a clean checkout for an END-TO-END regression: the
n=6 corpus (trajectories + .rallies.csv) is referenced only by the gitignored
output/human_lovo_manifest.json via absolute, machine-specific Windows paths; the
*_golden_features.json substrate lands in gitignored output/. So a 'video-free regression from
committed files' must FIRST commit a (small) frozen substrate + labels + a repo-relative
manifest. eval_baselines/ is the proven tracked home.",
    "regression.regress can only obtain PREDICTIONS via harness.get_predictions, which queries
the SQLite DB. There is no existing path to feed regression committed predictions from a file ?
a thin 'predictions-from-file' loader (or reusing experiment.VideoCandidates ->
evaluate_variant) must be ADDED so the gate runs with no DB.",
    "The two truth-sources can diverge: harness/regression score DB rows (candidate_segments /
play_segments) while experiment/ablation/served_gate_a score trajectory-derived windows. A file-
driven design should pick ONE substrate and record its provenance (windowing/operating-point,
detector, gt_hash) to avoid an incommensurable committed baseline.",
    "Regression is NOT a CI gate today (ci.yml runs only lint/mypy/import-
smoke/pytest+coverage). If the design intends CI enforcement, that wiring is net-new and must
run in the no-GPU lane.",
    "splits.assert_disjoint exists but is NOT wired into regression/distill_local ? committing
a golden fixture risks silent train/eval leakage unless the guard is connected when the fixture
is introduced.",
    "test_served_gate_a_regression.py and the golden-corpus tests skip cleanly when files are
absent; this means the production-path Gate-A litmus is effectively UNVERIFIED in CI ? a
committed mini-substrate would also let that litmus (or a reduced form) actually run in CI.",
    "A committed golden corpus must respect the licensing + minors-exclusion guardrails; raw
match footage cannot be committed, but derived trajectory CSVs / feature JSON / interval labels
are the safe, video-free artifacts to commit."
]
},
{
    "area": "Golden-set storage, sharing, and regression model ? design + built state",
    "summary": "Golden ground truth is per-video rally [start,end) interval CSVs, keyed by file

```

MD5 (video_id), produced by the Roora vendor (schema v8) and ingested via the sibling collector. The eval harness (metrics/regression/experiment) is structurally video-free already ? it re-scores predictions stored in SQLite and golden labels read from CSV. CRUX: NO doc proposes committing structured per-video label or feature artifacts to the repo for video-free regression. The deliberate design is the OPPOSITE: GOLDEN_DATA_SHARING.md keeps the heavy per-video artifacts (golden-label CSVs, *_golden_features.json, trajectory CSVs, manifests) OUT of git ? they live in gitignored output/ and sync between the GPU box and CPU dev sessions via a PRIVATE OneDrive folder, explicitly because they derive from the proprietary corpus and must never be committed. What IS tracked in git is only the small SCORE artifacts in eval_baselines/ (aggregate + per-video F1, deltas, honest CI/sign-test, label_set_hash, spec hashes) ? no raw labels or features. So a new proposal to commit structured per-video artifacts for video-free regression would EXTEND a gap, not duplicate anything; it must reconcile with (a) the OneDrive sharing convention and (b) the licensing/privacy guardrail that keeps proprietary-derived per-video data private.",

"keyFindings": [

"(1) HOW STORED/SHARED TODAY: Golden labels = one CSV per video, one rally per row, [start,end). Two schema layers: the minimal eval-harness format (EVALUATION.md §2: 'start,end' header optional; decimal seconds OR MM:SS/HH:MM:SS; malformed rows fail loudly) read by backend/eval/annotations.py + gt_loader.load_gt_intervals (the canonical golden reader). The richer VENDOR deliverable (ROORA_LABELING_GUIDELINES.md §1) is a per-rally CSV named exactly <video_id>.csv. Keying is by video_id = file MD5 (CLAUDE.md; EVALUATION.md §3; compute_video_id) ? the vendor is told NOT to trim/re-encode because the pipeline IDs by checksum. The golden-label naming convention in code is X.rallies.csv -> X.MP4 (fusion_golden.py::_derive_video_path).",

"(1b) LOCATION: live golden CSVs + the GPU-extracted *_golden_features.json + trajectory CSVs live in gitignored output/ (per-machine) and scattered local folders.

GOLDEN_DATA_SHARING.md (#175, 2026-06-15) wires GPU box <-> CPU dev sessions via a single PRIVATE OneDrive folder: %USERPROFILE%/OneDrive/rally-annotator/golden-data/ with subdirs features/<corpus-tag>/, trajectories/<corpus-tag>/, manifests/, baselines/. corpus-tag = e.g. 2026-06-15-n6. Raw videos are NEVER put there (kept local to GPU box). Optional ergonomics: RALLY_GOLDEN_DIR env var + scripts/sync_golden.py (in #175, not required).",

"(2) CRUX ? NO doc proposes committing structured per-video LABEL/FEATURE artifacts to the repo for video-free regression. The codebase ALREADY supports video-free regression a different way: detection runs ONCE per video, all per-cue signals + per-candidate features persist to SQLite candidate_segments.stats (JSON) and to output/*_golden_features.json, then ANY variant re-scores OFFLINE on CPU with no GPU/video (EXPERIMENT_HARNESS.md §2, §5.1; ABLATION_LAYER.md; standing lesson 'persist detection once, re-score forever'). But the persistence target is gitignored output/ + private OneDrive, NOT the repo. GOLDEN_DATA_SHARING.md is explicit: 'The shared folder lives OUTSIDE the repo -> never committed. Keep output/ gitignored... features derive from the Proprietary corpus -> keep the OneDrive folder private.' So committing per-video artifacts to git would be a NEW extension, contradicting the current private/OneDrive sharing posture unless reconciled.",

"(2b) WHAT IS COMMITTED TO THE REPO TODAY (verified via git ls-files): only eval_baselines/*.json SCORE artifacts ? experiment_leaderboard.json (per-A/B-comparison aggregate P/R/F1/mIoU + honest_delta_f1 CI + sign test + label_set_hash + variant spec_hash + promotion/revert records), heuristic_lovo_n6.json (mean + per_video F1 for the 6-match corpus), gemini_wrapper_2026-06-10.json. These contain SCORES and hashes, NOT raw rally labels or feature values. regression.py's DEFAULT_BASELINE_PATH eval_baselines/regression_baseline.json is referenced but NOT present/tracked. So the repo already commits a fingerprint+score layer (label_set_hash, gt_hash/gt_fingerprint) that ties a baseline to a label set WITHOUT storing the labels ? a natural anchor any new per-video-artifact proposal should build on.",

"(3) SCHEMA + VERSIONING (Roora): ROORA_LABELING_GUIDELINES.md is at VERSION v8 (2026-06-13), declared the SINGLE canonical copy and the sole label to use going forward; it supersedes Drive copies v3/v5/v6/v7 and an internal 'DRAFT v2' (those numbers never matched). The in-repo file is canonical because the fusion fields originate here; a sibling-collector mirror, if staged, must be regenerated from this file. Columns: rally_number, start_mmss, end_mmss, start_time(sec, auto), end_time(sec, auto), shots_count, ending_reason {winner|forced_error|unforced_error|service_fault|let|other}, last_shot_outcome {in|net|out|n_a}, score_before, score_after, notes. Plus a fusion-added per-video INTAKE MANIFEST (NOT in the per-rally CSV): video_id, fps, court_corners (4 px points), net_line (2 px endpoints), singles_or_doubles. ending_reason is shared across all net-separated sports. Three golden rules: ignore all dead time; include EVERY rally (incl. service faults/lets, EXCEPT warm-up and partial edge rallies); a shot = one contact. Quality bar: boundaries within ±0.3s, frame-grid boundaries auto-FAIL. Tier-B (per-shot timestamps, per-shot type) is appendix-only, do-not-start.",

"(4) BUILT vs PLANNED vs OWNER-GATED. BUILT: GS-1 FileProvider + AnnotationProvider registry (backend/annotations/, providers file+auto); GS-2 regression.regress()/regress_with_provider + save/load_baseline + run_regression.py CI CLI (exit 0/1/2/3); GS-4 AutomatedProvider oracle scaffold; the full eval engine (metrics.py, calibration.py LOVO, classifier.py pinnable JSON model, training.py label/feature glue); EXPERIMENT_HARNESS P1 experiment.py compare()/compare_unlabeled()/record_experiment (shipped 2026-06-13); ABLATION_LAYER Phase-0 ablation.py (LOSO/LOGO, CI+sign test, litmus test); fusion_golden.py GPU extractor -> *_golden_features.json; served_gate_a.py litmus; collector import-local for 3 sports; Roora v8 guideline. PLANNED/owner-gated: GS-3 HumanVendorProvider (parked on Roora pilot confirming quality); GS-5 trigger wiring (CI gate/scheduled); EXPERIMENT_HARNESS P2 per-cue flags+feature persistence, P3 CI promotion gate; PER_VIDEO_OUTPUT_LAYOUT (design only, not started); internal gold key still pending. GATE-A: PROMOTED then REVERTED (#146->#151, eval!=serve) ? now OFF-by-default, opt-in only.",

"(5) STATED CONSTRAINTS ON SHARING GOLDEN DATA. Licensing/privacy: golden sets built ONLY from licensed/owned footage (founder's collection + ShuttleSet); NO end-user uploads ever flow to a vendor; all third-party video egress through ONE provider boundary with DPA/no-reuse/delete-on-delivery/access-logging (GOLDEN_SET_VENDURING.md §2,§4). Repo: per-video golden artifacts are proprietary-derived -> kept OUT of git, private OneDrive only, no public share links (GOLDEN_DATA_SHARING.md Guardrails). Minors: EXCLUDED ENTIRELY from the labeled pool ? footage with children is not annotated, flagged back; minors-exclusion enforced at the pooling query (ROORA v8 §8 data-handling; QUALITY_ITERATIONS personalization decisions; CLAUDE.md). Biometrics/identity: no biometric/identity labeling, players referred to by per-video pseudonyms only, no cross-video person ID, no gait, no pose/skeleton/keypoint; work-for-hire IP assignment of labels (ROORA v8 §8). Train/eval leakage: split by match/session never by clip; hold out whole conditions; train+eval in separate catalogs with HARD ingest/export guardrails still a TODO/BACKLOG (GOLDEN_SET_PHASE1_VIDEOS §5)."

```

],
"dataStructures": [
  {
    "name": "Golden rally label CSV (eval-harness minimal format)",
    "where": "docs/EVALUATION.md §2; read by backend/eval/annotations.py +
gt_loader.load_gt_intervals; naming X.rallies.csv (fusion_golden.py)",
    "shapeOrFormat": "One CSV per video. Header 'start,end' optional/auto-detected. One
rally per row: start,end as decimal seconds (102.5) OR MM:SS/HH:MM:SS (00:01:12), forms mixable.
Blank lines and # comments ignored. Malformed rows (bad ts, start>=end) fail loudly. video keyed
by file MD5."
  },
  {
    "name": "Roora vendor per-rally CSV (schema v8)",
    "where": "docs/ROORA_LABELING_GUIDELINES.md §1; ingested via sibling collector python -m
src.cli.curate import-local",
    "shapeOrFormat": "One CSV per video named <video_id>.csv, UTF-8, header required, one
row per rally chronological. Cols: rally_number(int 1-based), start_mmss(m:ss.mmm), end_mmss,
start_time(sec auto), end_time(sec auto), shots_count(int),
ending_reason(winner|forced_error|unforced_error|service_fault|let|other),
last_shot_outcome(in|net|out|n_a), score_before, score_after, notes(required when other). Rules:
end>start; no overlaps end[N]<=start[N+1]; ±0.3s boundary tolerance."
  },
  {
    "name": "Per-video intake manifest (fusion-added)",
    "where": "docs/ROORA_LABELING_GUIDELINES.md §A; template INTAKE_MANIFEST_TEMPLATE.csv
(collector side)",
    "shapeOrFormat": "One row per video, recorded once (NOT in the per-rally CSV):
video_id(str), fps(num), court_corners(4 (x,y) px points clockwise from near-left), net_line(2
(x,y) px endpoints), singles_or_doubles(singles|doubles). Worked ex:
court_corners=[(360,980),(1560,980),(1330,300),(590,300)], net_line=[(470,640),(1450,640)]."
  },
  {
    "name": " *_golden_features.json (replayable offline re-scoring substrate)",
    "where": "written by backend/eval/fusion_golden.py to gitignored
output/<name>_golden_features.json; shared via OneDrive features/<corpus-tag>; NOT committed to
git",
    "shapeOrFormat": "Per-video JSON record: {name, fps, frame_width, candidates:[{start,
end, shuttle{5 fps/res-invariant feats:
duration,peak_velocity,mean_velocity,active_ratio,net_crossings}, person{5 feats:

```

players_in_court,two_sided_motion,mean_player_motion,in_court_ratio,shuttle_player_colocation},
label(int via IoU match to GT)]], gts}. Re-scored forever on CPU by fusion_compare/ablation ? no
GPU/video."

```
    },  
    {  
        "name": "candidate_segments table (SQLite persistence substrate)",  
        "where": "backend/infrastructure/database.py (add_candidate_segment :318,  
query_candidate_segments :355); output/<db_name>",  
        "shapeOrFormat": "Per-candidate row: video_id, detector, start_time, end_time,  
chunk_index, confidence, stats(JSON ? per-cue features), detection_params(JSON), human_label,  
confirmed_segment_id. The persist-once/re-score-forever substrate; keyed by video_id (file  
MD5)."  
    },  
    {  
        "name": "eval_baselines/experiment_leaderboard.json (TRACKED in git ? scores only)",  
        "where": "eval_baselines/experiment_leaderboard.json; written by  
experiment.record_experiment",  
        "shapeOrFormat": "JSON array of comparison/promotion records: {timestamp, iou,  
label_set_hash, num_videos,  
variant_a/b:{name,spec_hash,aggregate:{precision,recall,f1,mean_iou}}, delta:{...}, honest_delta  
_f1:{mean_delta,ci_low,ci_high,n,ci_excludes_zero,sign_test:{wins,losses,ties,n_effective,p_valu  
e}}, optional_promotion:{cue,promoted_to,decision,reverted,...}}. Contains SCORES + hashes, NO  
raw labels/features."  
    },  
    {  
        "name": "regression_baseline.json (per-video score snapshot)",  
        "where": "backend/eval/regression.py DEFAULT_BASELINE_PATH =  
eval_baselines/regression_baseline.json (referenced; not currently tracked);  
eval_baselines/heuristic_lovo_n6.json is a tracked sibling",  
        "shapeOrFormat": "save_baseline(video_id, stage, precision, recall, f1, ...,  
gt_fingerprint). RegressionResult: regressed, reasons, gt_hash, baseline P/R, tolerance,  
has_baseline. heuristic_lovo_n6.json: {name, description, computed, mean_f1, std,  
per_video:{<video_name>: f1}}. Per-video keyed by video_id/name; ties a baseline to a label set  
via gt_fingerprint/label_set_hash WITHOUT storing labels."  
    }  
],  
"videoDependency": "The eval/regression machinery is ALREADY video-free at score time and  
that is the explicit architecture: detection (GPU/WSL, needs the video) runs ONCE per video; all  
per-cue signals + per-candidate features persist (SQLite candidate_segments.stats JSON and  
output/*_golden_features.json); then fusion_compare.py / ablation.py / experiment.compare() /  
run_regression.py re-score ANY number of variants on pure CPU reading only the small JSON + the  
golden-label CSV ? no video, no GPU, no API calls, deterministic (EXPERIMENT_HARNESS $2,  
ABLATION_LAYER, EVALUATION $intro). Golden-label CSVs themselves require no video to consume.  
What is NOT yet video-free / not in the repo: the per-video artifacts (golden-label CSVs,  
*_golden_features.json, trajectory CSVs, manifests) are gitignored and shared only via private  
OneDrive (GOLDEN_DATA_SHARING.md), so a fresh git clone alone CANNOT run a regression ? it has  
the score baselines (eval_baselines/) and the harness code but not the labels/features. A clone  
CAN run the deterministic synthetic-row unit tests; the golden-litmus tests  
(test_ablation.py::test_litmus_reproduces_gate_a, test_fusion_compare.py) skipif the  
output/*_golden_features.json substrate is absent. So committing structured per-video artifacts  
to the repo is exactly what would make regression runnable from a clone alone ? the unfilled  
gap.",  
"builtVsPlanned": "BUILT (shipped): GS-1 AnnotationProvider ABC+registry+FileProvider  
(backend/annotations/, keys file+auto); GS-2 regression.regress()/regress_with_provider +  
save/load_baseline + run_regression.py CI CLI (exit 0/1/2/3); GS-4 AutomatedProvider oracle  
scaffold; eval engine  
metrics.py/calibration.py(LOVO,cross_validate,improvement_report)/classifier.py(pinnable JSON  
LogisticRegression)/training.py(label_candidates,build_training_set); EXPERIMENT_HARNESS P1  
experiment.py compare()/compare_unlabeled()/record_experiment + JSON leaderboard (2026-06-13);  
honest-stats helpers C1-C4 (eval_stats.py, score_calibration.py, segmentation_metrics.py ?  
landed, NOT yet wired into default harness flow); ABLATION_LAYER Phase-0 ablation.py with litmus  
test; fusion_golden.py GPU extractor -> *_golden_features.json; served_gate_a.py litmus + test;  
FusionSegmenter (fusion_hybrid) default-OFF; collector import-local for 3 sports; Roora v8  
guideline (in-repo canonical). PLANNED (not started): GS-3 HumanVendorProvider (parked until  
Roora pilot confirms quality); GS-5 trigger wiring; EXPERIMENT_HARNESS P2 per-cue enable flags +
```

feature persistence, P3 CI promotion gate; PER_VIDEO_OUTPUT_LAYOUT (L0-L3, design only);
internal gold key; hard train/eval-catalog ingest/export guardrails (BACKLOG);
GOLDEN_DATA_SHARING optional RALLY_GOLDEN_DIR + sync_golden.py. OWNER-GATED / NOTABLE: GATE-A
net_crossings>=2 was PROMOTED to served default (#146) then REVERTED (#151) as an eval!=serve
regression (offline +0.074 on proposal windowing did NOT transfer to coarse served windowing,
mean served ?F1 -0.008) ? now OFF-by-default, opt-in (indexing.fusion.min_crossings=2). Person
cue ?F1 -0.021 and +audio +0.079 both fail the promotion bar at n=6 (CIs span 0) -> stay
default-OFF, retained as feature columns. Whole golden-set/owned-model implementation track is
owner-gated; corpus = 6 labelled matches in output/human_lovo_manifest.json.",
"constraints": [
"Golden data built ONLY from licensed/owned footage (founder collection + ShuttleSet); NO
end-user uploads ever flow to a vendor (GOLDEN_SET_VENDORING \$2)",
"Per-video golden artifacts are proprietary-derived -> kept OUT of git; shared only via
PRIVATE OneDrive (no public links); output/ stays gitignored; do not symlink the shared folder
into a tracked path (GOLDEN_DATA_SHARING Guardrails)",
"Only eval_baselines/*.json SCORE artifacts (aggregate+per-video F1, deltas, CI/sign-test,
label_set_hash, spec hashes) are committed to the repo ? NOT raw labels or feature values",
"Minors EXCLUDED ENTIRELY from the labeled/training pool; footage with children is not
annotated and flagged back; minors-exclusion enforced at the pooling query (ROORA v8 \$8;
CLAUDE.md GDPR Art.9)",
"No biometric/identity/gait/pose/skeleton/keypoint labeling; players referenced by per-
video pseudonyms only, no cross-video person ID; labels are work-for-hire IP-assigned (ROORA v8
\$8)",
"Third-party video egress through ONE provider boundary with DPA, no-reuse, delete-on-
delivery, access logging; NDA before any footage shared (GOLDEN_SET_VENDORING \$4; ROORA \$8)",
"video_id = file MD5 is the immutable key ? vendor must NOT trim/re-encode/rename source
video or the checksum (hence video_id) changes (ROORA v8 \$1; EVALUATION \$3)",
"Train/eval split by match/recording-session never by clip; hold out whole conditions;
broadcast minimal in eval; HARD ingest/export guardrails so a video can't land in both catalogs
are still a TODO/BACKLOG (GOLDEN_SET_PHASE1_VIDEOS \$5)",
"Honest small-N contract: never a bare mean ? promote a cue only when ?F1 CI excludes 0
AND the sign test agrees; structural guards (Gate-A) exempt from auto-demotion (ABLATION_LAYER;
QUALITY_ITERATIONS)",
"Experiment infra is single-tenant/local-first: a JSON leaderboard the size of
baseline.json ? explicitly NOT MLflow/W&B/a service (EXPERIMENT_HARNESS \$9)",
"Roora v8 is the SINGLE canonical guideline (supersedes Drive v3/v5/v6/v7 + internal DRAFT
v2); the in-repo file is canonical, any collector mirror regenerated from it (ROORA header)",
"Fast-tier AutomatedProvider oracle MUST be a different model lineage than production
(else shared blind spots -> false green); a fast-tier regression is a smoke alarm that escalates
to the human tier (GOLDEN_SET_VENDORING \$5; GS-4)"
],
"relevantPaths": [
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_DATA_SHARING.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\GOLDEN_SET_IMPLEMENTATION.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_VENDORING.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\GOLDEN_SET_PHASE1_VIDEOS.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\EVALUATION.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\EXPERIMENT_HARNESS.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\ABLATION_LAYER.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\ROORA_LABELING_GUIDELINES.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\QUALITY_ITERATIONS.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\PER_VIDEO_OUTPUT_LAYOUT.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\eval_baselines\\experiment_leaderboard.json",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\eval_baselines\\heuristic_lovo_n6.json",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\eval_baselines\\gemini_wrapper_2026-06-10.json",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\fusion_golden.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\regression.py",


```

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\experiment.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\ablation.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\gt_loader.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\annotations.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\annotations\\base.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\infrastructure\\database.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\run_regression.py"
],
"risksGaps": [
  "CRUX RESTATED for the caller: do NOT assume an existing 'commit per-video artifacts to repo' proposal exists ? there is none. The only existing per-video-artifact sharing design is GOLDEN_DATA_SHARING.md, which deliberately keeps them OUT of git (private OneDrive). A new proposal would EXTEND this gap but must not CONTRADICT: (a) the 'never commit proprietary-derived data' guardrail, (b) the OneDrive sync convention, (c) the minors/biometric exclusions. Likely reconciliation: commit only DERIVED, de-identified, label-free or licensing-clean artifacts (e.g. the *_golden_features.json feature vectors are scale/translation-invariant counts/ratios, not raw frames or coords ? these MAY be repo-safe where raw CSVs/videos are not; confirm with owner).",
  "The repo already commits a fingerprint layer (label_set_hash in experiment_leaderboard.json; gt_fingerprint/gt_hash in regression.py) that binds a score baseline to a label set without storing labels. Any new per-video-artifact-in-repo design should build on these hashes for integrity/keying rather than inventing a parallel scheme.",
  "regression.py DEFAULT_BASELINE_PATH points at eval_baselines/regression_baseline.json but that file is NOT present/tracked ? run_regression.py would hit exit-3 (no-baseline) on a fresh clone. The tracked baselines are experiment_leaderboard.json + heuristic_lovo_n6.json + gemini_wrapper_*.json (different shape). A clone cannot currently run the per-video regression gate end-to-end.",
  "Golden-litmus tests skipif the output/*_golden_features.json substrate is absent ? so on CI / a fresh clone the golden-litmus coverage is silently skipped; only synthetic-row unit tests run. Committing the (de-identified) feature substrate would also light up these tests in CI ? a concrete motivation for the proposal.",
  "Two schema layers coexist (minimal 'start,end' eval CSV vs the rich Roora <video_id>.csv). The collector ingests the rich one; the indexer harness reads the minimal one via gt_loader. Any artifact-in-repo proposal must be explicit about which schema it freezes and how the rich->minimal projection is captured.",
  "Train/eval leakage hard-guardrails (separate catalogs enforced at ingest/export) are still only a convention + BACKLOG TODO ? a video could currently land in both train and eval by mistake; relevant if committed artifacts blur the train/eval boundary.",
  "Gate-A eval!=serve revert is the cautionary tale baked into the leaderboard: a number measured on the harness windowing did not transfer to the served windowing. Any regression model committed to the repo must record WHICH windowing/operating-point the baseline was measured at, or it will mispredict served behaviour."
]
},
{
  "area": "Pipeline data contracts and video-free seam(s) for regression",
  "summary": "The pipeline is a clean 4-stage chain: (1) DETECTION ? the WSL shuttle detector (WASB/TrackNet) reads frames and emits a per-frame trajectory CSV (`Frame,Visibility,X,Y`); (2) WINDOWING ? `TrackNetRunner.trajectory_to_action_windows` (the model-agnostic `\"brain\"`) turns the trajectory into candidate rally window dicts, persisted to `candidate_segments`; (3) ADJUDICATION/HANDOFF ? fusion gates + AI provider confirm windows into `play_segments`; (4) STITCH ? `VideoCompiler.compile_highlights` re-encodes/concat the selected `(start,end)` time-pairs into the reel. Video frames are required in exactly TWO places: the WASB/TrackNet trajectory pass (Stage 1) and the person-detector pass (fusion P2, default-OFF) ? both consume the original video by absolute frame index. EVERYTHING from the trajectory CSV onward (windowing, net-crossing Gate A, person features given boxes, candidate persistence, IoU scoring, segment selection, merge) is PURE deterministic transformation. The provider handoff (Gemini) and the final ffmpeg stitch are the only other steps that touch the media file. The single best video-free regression seam is the TRAJECTORY CSV: freeze `<key>_wasb.csv` per video and the windowing?gating?scoring chain replays with no GPU/WSL/frames. A second, complementary seam is the persisted `candidate_segments` rows (post-windowing), and for fusion the per-frame person-detection dict.",
  "keyFindings": [
    "STAGE SEQUENCE (runtime, trajectory hybrids = the production shuttle path): config load

```

```

-> video_id=MD5(file) (backend/utls/hashing.py::compute_video_id, 64KB chunks) -> validators ->
db.add_video (UPSERT) -> segmenter.process_video. Inside
TrajectoryHybridSegmenter.process_video: P1 healthcheck gate -> P2
runner.run_predict(video,out_dir='output/<family>',video_id) writes trajectory CSV;
parse_trajectory_csv -> List[TrajectoryPoint]; trajectory_to_action_windows -> candidate window
dicts; _enrich_candidate_stats per window -> window_stats; db.add_candidate_segment persists
each (full superset) -> P3 _select_for_handoff picks indices; for each,
extract_video_chunk(reencode=True) padded clip -> provider.analyze_video(clip,prompt) -> AI
segment dicts (or skip_ai => windows emitted verbatim) -> P4 db.add_play_segment +
link_candidate_to_segment. Then COMPILE (separate call): db.query_play_segments -> filter by
stitching.min_shot_count -> time_pairs=[(start_time,end_time)] ->
VideoCompiler.compile_highlights(filepath,time_pairs,out_name).",
    "ARTIFACT 1 ? Trajectory CSV (Stage-1 output, the natural freeze seam): file
`output/<family>/<stem>_<video_id[:12]>.wasb.csv` (WASB) or `<key>_predict.csv` (TrackNet).
Exact columns: `Frame,Visibility,X,Y` (header + order fixed; Visibility==1 means visible; X,Y
are ORIGINAL-frame pixel coords). Parsed to TrajectoryPoint{frame:int, visible:bool, x:float,
y:float}, frame-sorted. This is the complete handoff from the GPU/WSL world to the pure-Python
world.",
    "ARTIFACT 2 ? Candidate window dict (output of trajectory_to_action_windows; in-memory):
{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int (==1 for shuttle trajectory), chunk_index:Optional[int]
(==0 for whole-video trajectory pass)}. Windowing is deterministic: active frame iff visible AND
normalized per-second velocity (dist/frame_width * fps/dframes) > velocity_thresh; active frames
within merge_gap*fps grouped; windows shorter than min_window_duration dropped. fps<=0 ->30.0
and frame_width<=0 ->1280.0 silent fallbacks. visible==False resets prev (occlusion breaks the
track).",
    "ARTIFACT 3 ? candidate_segments DB row (persisted, detector-agnostic fine-tuning table;
THE second freeze seam): columns id, video_id, detector:str, chunk_index, start_time:REAL,
end_time:REAL, duration:REAL(=end-start), confidence:REAL(=min(1.0,peak_velocity)), stats:JSON,
detection_params:JSON, confirmed_segment_id(FK SET NULL), human_label, notes, created_at,
user_id(NULL=local). stats JSON (base trajectory) = {peak_velocity, mean_velocity,
active_frame_count, track_id_count}. detection_params by detector: wasb={detector,velocity_thres
h,merge_gap,min_action_window_duration,weights_path,fusion_substrate?}; tracknet adds
queue_length; yolo={velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector_score_t
hreshold,min_action_window_duration}.",
    "PER-SIGNAL FEATURE COLUMNS ? Gate A (net-crossings):
FusionSegmenter._enrich_candidate_stats ALWAYS adds stats['net_crossings']:float via
rally_gate.gate_windows (pure, from the trajectory; axis/net estimated once per video, cached).
Person cue (default-OFF, only if indexing.fusion.cue_flags.person): adds the 5
PERSON_FEATURE_NAMES = players_in_court, two_sided_motion, mean_player_motion, in_court_ratio,
shuttle_player_colocation (all scale/translation-invariant floats from
fusion_features.compute_person_features). Court mask = optional normalized
indexing.fusion.court_polygon; net=(net_axis,net_position). The learned-classifier shuttle
features (distill_local.FEATURE_NAMES) = duration, peak_velocity, mean_velocity, active_ratio,
net_crossings; the Gen-0 per-frame schema (backend/features/rally_seq.py FEAT_NAMES) = vis_rate,
spd_mean, spd_max, cross_rate, x_std, y_std.",
    "ARTIFACT 4 ? play_segment dict (final segment record, db.add_play_segment /
query_play_segments): {id:int, video_id, start_time:float, end_time:float, duration:float(=end-
start), server, receiver, metadata:dict, events:[...]}. Hybrid P4 metadata = {shot_count:int
(provider shuttle_exchange_count else max(3,int(dur/0.8))), shot_count_confidence:str (provider
value or 'Unknown'/'LocalNoAI'), detector:f'{family}-hybrid-{model_name}'}. Provider (AI)
segment dict consumed by P3 = {start_time, end_time, shuttle_exchange_count?,
shot_count_confidence?} (timestamps offset by clip w_start; _candidate_id stamped internally).",
    "FINAL REEL transform (VideoCompiler.compile_highlights): segments=[(start,end)] ->
_merge_overlapping (parse times, drop end<=start, sort, merge overlapping/abutting ranges so a
rally is never stitched twice) -> per-clip ffmpeg re-encode (libx264/superfast/crf22, identical
settings, optional watermark via -vf) -> `-f concat -c copy` -> output/<name>.mp4. Selection
upstream: API filters by stitching.min_shot_count (segments without shot_count are exempt) and
by req.segment_ids, then time_pairs=[(s['start_time'],s['end_time'])].",
    "TWO ALTERNATE PATHS that bypass the trajectory seam: `gemini` segmenter (pure-AI, no
P2/candidates ? chunks the whole video, uploads, provider returns segments directly to
play_segments) and `motion` (pixel-absdiff baseline with FABRICATED random metadata/events).
These two are the _FINAL_ONLY_SEGMENTERS in eval/batch.py ? they have no candidate stage and
cannot be frozen at the trajectory seam. The video-free seam analysis applies to the trajectory
hybrids (wasb_hybrid/tracknet_hybrid/fusion_hybrid), which are the rally-quality track."

```

```

],
"dataStructures": [
  {
    "name": "Trajectory CSV",
    "where": "output/<family>/<stem>__<video_id[:12]>_wasb.csv (WASB) / <key>_predict.csv
(TrackNet); written in WSL, read by
backend/pipeline/detectors/tracknet_runner.py::parse_trajectory_csv",
    "shapeOrFormat": "CSV header `Frame,Visibility,X,Y` (fixed order); rows: Frame:int,
Visibility:int(1=visible), X:float px, Y:float px ->
TrajectoryPoint{frame:int,visible:bool,x:float,y:float} frame-sorted"
  },
  {
    "name": "Candidate window dict",
    "where": "output of TrackNetRunner.trajectory_to_action_windows (tracknet_runner.py);
in-memory, consumed by trajectory_hybrid P2/P3",
    "shapeOrFormat": "{start:float(sec), end:float(sec), active_frame_count:int,
peak_velocity:float, mean_velocity:float, track_id_count:int(==1),
chunk_index:Optional[int](==0)}"
  },
  {
    "name": "candidate_segments row",
    "where": "backend/infrastructure/database.py (add_candidate_segment /
query_candidate_segments), table candidate_segments",
    "shapeOrFormat": "id, video_id, detector:str, chunk_index, start_time:REAL,
end_time:REAL, duration:REAL, confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count[,net_crossings,+5
person feats]}, detection_params:JSON, confirmed_segment_id, human_label, notes, created_at,
user_id"
  },
  {
    "name": "Per-signal feature columns (fusion)",
    "where": "backend/pipeline/detectors/fusion_features.py::PERSON_FEATURE_NAMES +
rally_gate net_crossings; persisted into candidate_segments.stats by
fusion_hybrid.py::_enrich_candidate_stats",
    "shapeOrFormat": "net_crossings:float (Gate A, always); person (flag-gated):
players_in_court, two_sided_motion, mean_player_motion, in_court_ratio,
shuttle_player_colocation (all floats, scale/translation-invariant)"
  },
  {
    "name": "Person per-frame detections",
    "where": "backend/pipeline/detectors/person_detector.py::detections_per_frame (fusion
P2, needs video+GPU)",
    "shapeOrFormat": "{ 'frames': {frame_idx:int -> [(track_id:int, bbox:(x1,y1,x2,y2)),
...]}, 'fps':float, 'frame_width':float, 'frame_height':float}; frame_idx aligns 1:1 with
trajectory .frame"
  },
  {
    "name": "play_segment dict (final)",
    "where": "backend/infrastructure/database.py::add_play_segment / query_play_segments,
table play_segments",
    "shapeOrFormat": "{id:int, video_id, start_time:float, end_time:float,
duration:float(=end-start), server, receiver,
metadata:dict{shot_count:int,shot_count_confidence:str,detector:'<family>-hybrid-<model>'},
events:[...]}"
  },
  {
    "name": "AI/provider segment dict",
    "where": "provider.analyze_video output, consumed by trajectory_hybrid P3 and
gemini.py",
    "shapeOrFormat": "{start_time:float(decimal sec), end_time:float,
shuttle_exchange_count?:int, shot_count_confidence?:'Low'|'Medium'|'High'}; +_candidate_id
stamped internally; times offset by clip start"
  },
  {
    "name": "Final reel time_pairs",

```

```

    "where": "backend/main.py::compile_highlights -> VideoCompiler.compile_highlights
(compiler.py)",
    "shapeOrFormat": "List[Tuple[start:float, end:float]] -> _merge_overlapping (sort+merge)
-> per-clip reencode + concat -c copy -> output/<name>.mp4"
},
{
    "name": "GT / golden CSV (eval comparison)",
    "where": "backend/eval/annotations.py::load_annotations (start,end) and
calibrate_wasb.load_golden_gts (rally_number,start_time,end_time)",
    "shapeOrFormat": "generic: `start,end` 2-col (RAISES on bad row); golden/slicer:
`rally_number,start_time,end_time[,duration,start_mmss,end_mmss]` (SILENTLY skips bad rows)"
}
],
"videoDependency": "Video/frames are REQUIRED in exactly two compute steps, both at the
front of the trajectory-hybrid pipeline: (A) the shuttle trajectory pass ?
WasbRunner/TrackNetRunner.run_predict stages the video into WSL and runs the GPU CNN over
decoded frames to emit the `Frame,Visibility,X,Y` CSV (backend/pipeline/detectors/wasb_runner.py
+ wasb_infer.py, GPU/WSL/cv2); and (B) the person-detector pass ?
PersonDetector.detections_per_frame seeks the ORIGINAL video by absolute frame range (fusion P2,
default-OFF, torch/cv2/GPU). In addition the media file is touched (not 'frames' analysis but
read+transcode) at the AI handoff (extract_video_chunk re-encodes a padded clip and uploads to
Gemini) and at the final stitch (VideoCompiler re-encodes/concats clips with ffmpeg). EVERYTHING
ELSE is pure deterministic Python on numbers/strings: parse_trajectory_csv,
trajectory_to_action_windows (windowing brain), rally_gate (net-crossings Gate A),
fusion_features (person features given boxes ? pure, no cv2/torch), db.add_candidate_segment /
add_play_segment persistence, eval/metrics IoU scoring, _select_for_handoff gating, and
_merge_overlapping. So once the trajectory CSV (and, for fusion, the per-frame person dict)
exists, no video, GPU, WSL, ffmpeg, or cloud is needed to reproduce windowing, gating, candidate
rows, and IoU scores.",
"builtVsPlanned": "BUILT and live: full trajectory-hybrid path (wasb_hybrid is the live
shuttle substrate), parse_trajectory_csv + trajectory_to_action_windows (pure, static, model-
agnostic), candidate_segments persistence (schema v5), play_segments, VideoCompiler stitch with
overlap-merge, rally_gate.py (Gate A net-crossings ? PURE, unit-tested), fusion_features.py
(PURE person-feature math, unit-tested), FusionSegmenter (registered fusion_hybrid, default-OFF,
two identity hooks; net_crossings always persisted, person features + served Gate A/B flag-gated
default-0/OFF), eval harness (metrics/calibration/distill_local/rally_seq_proto), the
skip_ai_handoff fully-local mode. The eval drivers ALREADY consume the trajectory CSV directly
and replay windowing/gating/scoring with no video ? i.e. the video-free seam is de-facto
operational in eval/ (calibrate_wasb.make_predict_fn parses the trajectory once and closes over
trajectory_to_action_windows; eval_partial_labels, export_wasb_segments, calibrate_local all do
this). PLANNED/owner-gated: per-video output layout (PER_VIDEO_OUTPUT_LAYOUT.md, NOT started ?
trajectory dir still hardcoded 'output/<family>'), P3 learned fusion (person features into
classifier, gated on golden labels), P4 audio/pose cues, served Gate-A-in-production promotion
(thresholds default 0). NO formal frozen-fixture regression harness keyed on the trajectory CSV
exists as a tracked test yet ? the seam is exploited ad-hoc by eval drivers but not pinned as a
video-free regression suite.",
"constraints": [
    "Trajectory CSV columns are FIXED: `Frame,Visibility,X,Y` header + order; X,Y are
ORIGINAL-frame pixels (affine-resized back from the 512x288 detector input). A frozen fixture
must capture exactly this.",
    "Windowing needs fps + frame_width alongside the CSV (it normalizes velocity by
frame_width and converts frame->sec by fps). _probe_fps_width reads these from the video via
cv2; for a video-free replay they MUST be captured/pinned (fallbacks 30.0/1280.0 are silent and
would skew results). This is the one non-CSV input the windowing brain needs.",
    "Person features need the per-frame detection dict {frame_idx -> [(track_id,bbox)]} +
frame_width/height; frozen separately from the CSV. Their frame indices must align 1:1 with the
trajectory .frame.",
    "Candidate persistence is detector-agnostic by design (stats/detection_params JSON) ? a
frozen candidate_segments row is a valid second seam but is one transform DOWNSTREAM of the CSV
(windowing already applied), so it cannot regress the windowing brain itself.",
    "gemini and motion segmenters have NO candidate/trajectory stage (final-only) ? they
cannot use the trajectory seam; their regression must freeze provider responses or accept they
touch video.",
    "eval/metrics.match_intervals is GREEDY (not Hungarian) and deterministic; empty matches
yield 0.0 not NaN ? stable for golden assertions.",

```

"License guardrail: any frozen-fixture tooling must stay MIT/Apache/BSD; WASB weights are academic-data-trained (confirm terms before commercial deploy)."

```
],
"relevantPaths": [
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\tracknet_runner.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\wasb_runner.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\base.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\rally_gate.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\fusion_features.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\person_detector.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\segmenters\\trajectory_hybrid.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\segmenters\\fusion_hybrid.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\segmenters\\base.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\infrastructure\\database.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\stitcher\\compiler.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\utils\\hashing.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\metrics.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\calibrate_wasb.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\harness.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\features\\rally_seq.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\main.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\CODE_MAP.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\HOW_RALLY_DETECTION_WORKS.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\MULTI_SIGNAL_FUSION_PLAN.md"
],
```

```
"risksGaps": [
  "SOURCES OF NON-DETERMINISM downstream of the trajectory-CSV seam ? these must be controlled or excluded from frozen assertions: (1) FLOAT ? velocity = (dist/frame_width)*(fps/dframes), median/spread in rally_gate, normalized person motion: all float ops, platform-stable but assert with tolerance not exact equality; (2) ORDERING/THREADING ? P3 uses ThreadPoolExecutor(max_workers=ai_max_workers); ai_segments.extend(future.result()) iterates futures in submission order so it IS deterministic, but the underlying provider calls are not (network) ? freeze at the windowing/candidate seam BEFORE P3, or stub the provider; (3) TIMESTAMPS ? candidate_segments/play_segments rows carry created_at DEFAULT CURRENT_TIMESTAMP and the DB autoincrement id ? exclude these columns from any golden comparison; (4) DB id NON-DETERMINISM ? play_segment.id is AUTOINCREMENT, link_candidate_to_segment writes it ? compare on (start_time,end_time,stats) not id; (5) ffmpeg/codecs ? the final reel bytes are NOT reproducible across ffmpeg builds (libx264 version, NVENC vs libx264 fallback, watermark font) ? never assert reel bytes; assert the time_pairs + merge output instead.",
  "fps/frame_width are probed from the VIDEO (cv2) inside the segmenter, not stored in the CSV ? a video-free replay that forgets to pin them silently falls back to 30.0/1280.0 and produces different windows. The frozen fixture MUST include the (fps, frame_width) used.",
  "No tracked frozen-fixture regression test currently pins the trajectory-CSV -> windows -> candidates -> IoU chain end-to-end as a video-free suite; the seam is only exercised ad-hoc by eval CLIs. This is the gap a regression harness would fill.",
  "trajectory_to_action_windows treats visible==False as a hard track break (occlusion gap resets prev) and silently clamps fps<=0->30 / width<=0->1280 ? a fixture with degenerate values would mask real regressions.",
  "Two segmenters (gemini, motion) bypass the seam entirely (no candidate stage); motion additionally fabricates RANDOM metadata/events (random.sample/uniform) so it is inherently non-
```

deterministic and unsuitable for golden assertions.",

"The person-feature path (fusion, default-OFF) needs a SEPARATELY frozen per-frame detection dict; cv2 frame-seek accuracy varies by codec ($\pm$ a few frames on real footage), so the captured dict must come from the same decode the fixture pins."

```
]
},
{
  "area": "Test suite execution model, committed fixture data, mocking conventions, and the
right home for new auto-discovered per-video regression fixtures",
  "summary": "Tests are pure-pytest auto-discovery (no test runner framework wrapping):
`pytest.ini` sets `testpaths = tests`, `run_all_tests.py` is just a thin `subprocess` wrapper
around `python -m pytest` (defaults to `-q`, passes all args through, prints a PASS/FAIL banner,
returns pytest's exit code). There is NO conftest.py and NO committed CSV/JSON test fixtures
under `tests/` or `backend/eval/` ? every test synthesizes its fixtures inline into pytest's
`tmp_path` (CSVs via `csv.writer`/`p.write_text`, manifests + golden-feature JSON via
`json.dump`). The only committed data files are 3 small JSONs in `eval_baselines/` (the no-
regression baselines/leaderboard, tracked deliberately) plus `config.json` and
`artifacts/rally_tal_gen0/0.1.0/{manifest,weights}.json`. The real per-video golden corpus
(trajjectory CSVs, *.rallies.csv labels, *golden_features.json, human_lovo_manifest.json) lives
ONLY in the gitignored `output/` dir on the owner's box ? it is NOT committed, so any test
depending on it (test_served_gate_a_regression.py) skips cleanly when absent. There IS a clean
load?transform?assert pattern to follow (fusion_compare.load_videos reads
*_golden_features.json), and an existing manifest-driven, skip-if-absent per-video regression
test to mirror (test_served_gate_a_regression.py). New committed, auto-discovered per-video
fixtures + a parametrized test should slot into a NEW fixtures dir UNDER tests/ (e.g.
tests/fixtures/golden/) with the parametrized test in tests/test_*.py globbing that dir.",
  "keyFindings": [
    "DISCOVERY/RUN: pytest.ini (repo root) is the only pytest config ? a single line
`testpaths = tests`. No pyproject [tool.pytest], no markers registered, no conftest.py anywhere
in the repo. run_all_tests.py (repo root, 60 lines) defaults to `-q`, passes every CLI arg
straight through to `python -m pytest`, and propagates pytest's exit code (CI/pre-PR friendly).
The full-suite lane enforces a 70% coverage floor (CODE_MAP $0); run with `python
run_all_tests.py --cov=backend --cov-report=term-missing`.",
    "SKIP/OPTIONAL-DEP CONVENTIONS: two idioms in use ? (a) `pytest.importorskip('cv2')` /
`importorskip('obsreport')` at module top to skip whole modules when a heavy/optional dep is
absent (CI lacks GPU/torch/cv2); (b) a module-level `_skip = pytest.mark.skipif(not _SPECS,
reason=...)` decorator gated on whether real golden artifacts exist on disk. These are how real-
video/GPU tests degrade to SKIP rather than ERROR in CI.",
    "NO COMMITTED TEST FIXTURES: `Glob tests/**/*.csv`, `tests/**/*.json`,
`backend/eval/**/*.csv|json` all return NOTHING. Every fixture is built inline into `tmp_path`.
CSV GT is written with `csv.writer` or `p.write_text('start,end\\n0,10\\n...')`
(test_gt_loader.py, test_eval_regression.py). Per-video golden-feature JSON is written with
`json.dump({name, fps, frame_width, candidates:[{start,end,shuttle},{person},{},label]},
gts:[[a,b]], ...) (test_fusion_compare.py::write_video). Manifests are `json.dump([name,
trajectory, fps, frame_width, labels]), ...) (test_fusion_golden_telemetry.py::write_manifest).",
    "COMMITTED DATA (the only tracked non-code data): eval_baselines/heuristic_lovo_n6.json
(689 B, 15 lines ? the LOVO floor + per_video f1 map),
eval_baselines/experiment_leaderboard.json (4.7 KB, 139 lines ? A/B variant rows with
honest_delta_f1 CIs), eval_baselines/gemini_wrapper_2026-06-10.json (1.98 KB). Plus config.json
(2.7 KB) and artifacts/rally_tal_gen0/0.1.0/{manifest,weights}.json (untracked here, ADR-008:
manifest committed / weights gitignored). eval_baselines/ is deliberately tracked so a clean
checkout still has the no-regression baselines.",
    "MOCKING ? DETECTOR: injected, not patched. extract_video(spec, detector=None,...) takes
an optional `detector` with a `detections_over_windows(video, windows, frame_skip, progress_cb)`
duck-typed interface; tests pass a `_StubDetector` returning canned `{frames, fps, frame_width,
frame_height}` dicts (test_fusion_compare.py). fusion_golden.run() is tested by monkeypatching
`fusion_golden.extract_video` wholesale.",
    "MOCKING ? WSL: patch the single seam method `_wsl_bash` on a WasbRunner instance with
`unittest.mock.patch.object(r, '_wsl_bash', side_effect=[_proc(rc,stdout,stderr), ...])`, where
`_proc` is a `types.SimpleNamespace(returncode, stdout, stderr)` stand-in for a
CompletedProcess. Higher-level seams `_stage_video`, `_stage_infer_script`, `_wsl_rm_rf`,
`run_predict` are also patched. No real `wsl.exe` is ever invoked (test_wasb_runner.py).",
    "MOCKING ? VIDEO/GPU: no real MP4 decode in the suite. fusion_golden derives the MP4 path
from the labels filename (X.rallies.csv -> X.MP4) but the stubbed detector never opens it.
```

TrackNetRunner.parse_trajectory_csv (Frame,Visibility,X,Y DictReader) is monkeypatched to return canned TrajectoryPoint lists, OR fed a tiny real CSV string. RunTelemetry (nvidia-smi/psutil) is made hermetic by monkeypatching gpu_name + injecting deterministic gpu_fn/sys_fn/clock collectors (test_fusion_golden_telemetry.py).",

"EXISTING load?transform?assert PATTERN TO FOLLOW:

backend/eval/fusion_compare.py::load_videos(features_dir) globs `\*\_golden\_features.json`, loads each into experiment.VideoCandidates, and compare() runs honest LOVO; tests assert on the resulting ?F1/CI/sign-test (test_fusion_compare.py). This is the canonical 'load artifact -> run transform -> assert' shape, but today the artifacts are written to tmp_path inside the test, never committed.",

"EXISTING PER-VIDEO REGRESSION TEST TO MIRROR: tests/test_served_gate_a_regression.py is a manifest-driven, per-video, skip-if-absent regression litmus. It resolves MANIFEST from ``<repo_root>/output/human_lovo_manifest.json`` (env-overridable via SERVED_GATE_A_MANIFEST), `\_golden\_specs()` keeps only specs whose trajectory+labels files exist, and `\_skip = skipif(not \_SPECS, ...)` skips the whole module in CI where the gitignored corpus is absent. It loops over each golden video and asserts served-path F1/over-seg invariants. This is the closest template for new per-video regression fixtures ? the gap a committed-fixture approach would close is that it currently CANNOT run in CI (corpus uncommitted).",

"GT LOADER CONTRACT (load-bearing for fixtures):

backend/eval/gt_loader.py::load_gt_intervals reads BOTH golden-CSV conventions ? header `start,end` (collector curate export) OR `start\_time,end\_time` (rally-annotator .rallies.csv, extra cols ignored), else positional cols 0,1; timestamps accept decimal seconds or MM:SS/HH:MM:SS; strict=True raises with file:line, strict=False skips+logs. Any committed CSV fixture must conform to one of these two shapes."

```
],
"dataStructures": [
{
    "name": "Per-video golden-feature JSON (*_golden_features.json)",
    "where": "Written by backend/eval/fusion_golden.py::extract_video/run into gitignored output/; consumed by backend/eval/fusion_compare.py::load_videos; synthesized inline in tests/test_fusion_compare.py::write_video",
    "shapeOrFormat": "{ \"name\": str, \"fps\": float, \"frame_width\": float, \"candidates\": [{ \"start\": float, \"end\": float, \"shuttle\": {5 SHUTTLE_FEATURE_NAMES: float}, \"person\": {5 PERSON_FEATURE_NAMES: float}, \"label\": int(0|1)}], \"gts\": [[start,end], ...]}. Tiny (a handful of candidates per video in synthetic tests). This is the natural unit for a committed per-video regression fixture."
},
{
    "name": "LOVO manifest (human_lovo_manifest.json)",
    "where": "Lives only in gitignored output/; consumed by fusion_golden.run, served_gate_a, calibrate_local; synthesized inline in tests/test_fusion_golden_telemetry.py::write_manifest",
    "shapeOrFormat": "JSON array of specs: [{ \"name\": str, \"trajectory\": \"<name>.csv\", \"fps\": float, \"frame_width\": float, \"labels\": \"<name>.rallies.csv\", optional \"video_path\": str}]"
},
{
    "name": "Trajectory CSV (TrackNet/WASB predict output)",
    "where": "backend/pipeline/detectors/tracknet_runner.py::parse_trajectory_csv (DictReader); referenced by manifest 'trajectory' field; only exists in gitignored output/",
    "shapeOrFormat": "CSV header `Frame,Visibility,X,Y`; one row per frame; Visibility==1 means shuttle visible. Parsed into TrajectoryPoint(frame:int, visible:bool, x:float, y:float)."
},
{
    "name": "Golden GT / rally-label CSV (two conventions)",
    "where": "backend/eval/gt_loader.py::load_gt_intervals (canonical), annotations.load_annotations, calibrate_wasb.load_golden_gts; synthesized inline in test_gt_loader.py / test_eval_regression.py",
    "shapeOrFormat": "Either header `start,end` (collector export) OR `rally_number,start_time,end_time,ending_reason,sport` (rally-annotator .rallies.csv), OR headerless positional cols 0,1. Timestamps: decimal seconds or MM:SS/HH:MM:SS. -> List[(start:float, end:float)]"
},
{
    "name": "Eval baseline / leaderboard JSON",
```

```

    "where": "Committed in eval_baselines/; written by
backend/eval/experiment.py::record_experiment + regression.save_baseline; read by
run_regression.py no-regression gate",
    "shapeOrFormat": "heuristic_lovo_n6.json: {name, description, computed, mean_f1, std,
per_video:{vid:f1}}. experiment_leaderboard.json: array of {timestamp, iou, label_set_hash,
num_videos, variant_a/b:{name,spec_hash,aggregate:{precision,recall,f1,mean_iou}}, delta,
honest_delta_f1:{mean_delta,ci_low,ci_high,n,ci_excludes_zero,sign_test}}",
    },
    {
        "name": "WSL CompletedProcess stand-in (_proc)",
        "where": "tests/test_wasb_runner.py",
        "shapeOrFormat": "types.SimpleNamespace(returncode:int, stdout:str, stderr:str) ? fed
via patch.object(runner, '_wsl_bash', side_effect=[...]) to mock every WSL bash call"
    }
],
    "videoDependency": "No test in the suite decodes a real video. The pipeline cannot run here
(dev container lacks GPU/torch/cv2/numpy per CLAUDE.md). Real-video/GPU paths are handled three
ways: (1) inject a stub detector with a duck-typed detections_over_windows() so no MP4 is
opened; (2) monkeypatch TrackNetRunner.parse_trajectory_csv / fusion_golden.extract_video to
return canned data; (3) for tests that genuinely need the real golden corpus (trajectory CSVs +
.rallies.csv labels + MP4s), gate the whole module behind pytest.importorskip('cv2') and/or a
skipif on whether the gitignored output/human_lovo_manifest.json and its referenced files exist
on disk ? these SKIP in CI and only run on the owner's box. The real 6-video golden corpus
(adarsh_avi_singles, kushagra_singles, largetest_doubles, gbaaddy,
testlarge_short/largetest_short, mahadevpura_singles) is NOT committed; only their derived
*_golden_features.json sit in gitignored output/.",
    "builtVsPlanned": "BUILT: full pytest auto-discovery suite (~85 test_*.py files), the
run_all_tests.py wrapper, the load?transform?assert pattern (fusion_compare.load_videos), the
inline-tmp_path fixture idiom, the skip-if-corpus-absent per-video regression litmus
(test_served_gate_a_regression.py), and the committed eval_baselines/ no-regression baselines.
NOT BUILT / GAP the task targets: there are currently NO committed per-video regression fixtures
and NO parametrized test that auto-discovers committed per-video artifacts ? the existing per-
video regression test depends on the uncommitted gitignored output/ corpus, so it never runs in
CI. A new tests/fixtures/golden/ dir of small committed *_golden_features.json + a parametrized
test would make per-video regression CI-runnable for the first time. (No conftest.py exists yet
either, so shared fixture plumbing would be a new addition.)",
    "constraints": [
        "Commercial-clean licensing + privacy: committed fixtures must EXCLUDE minors from any
training/eval pool and must not embed paid-LLM (Gemini) outputs as a training data source
(CLAUDE.md guardrails). Real-player footage frames cannot be committed; synthetic or trajectory-
only/feature-only JSON is the safe form.",
        "Fixtures must be TINY and offline: the suite is fully mocked, no GPU/WSL/network, and CI
has a 70% coverage floor. Committed *_golden_features.json should be small (a handful of
candidates), like the synthetic ones in test_fusion_compare.py ? not the full real corpus.",
        "CSV fixtures MUST conform to one of the two gt_loader conventions (header start,end |
start_time,end_time, or positional) or they will be rejected/skipped by load_gt_intervals
(strict mode raises with file:line).",
        "File-placement rule (CODE_MAP §0): tests go in tests/test_*.py; eval baselines go in
tracked eval_baselines/; pipeline run outputs go in gitignored output/. There is no existing
tests/fixtures or tests/data dir ? adding one is a new convention but consistent with 'a test ->
tests/'. Do NOT put fixtures in output/ (gitignored, won't survive checkout/CI).",
        "Branch from origin/master, stage explicit paths (no git add -A), re-verify HEAD before
commit ? shared checkout / concurrent sessions (CLAUDE.md + memory gotcha-shared-checkout-
branch-collisions).",
        "MANIFEST.in / hatchling packaging: pyproject lists packages=[backend, training]; a
tests/fixtures dir is fine for pytest (run from source tree) but would not be packaged into the
wheel ? acceptable since tests aren't shipped."
    ],
    "relevantPaths": [
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\run_all_tests.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\pytest.ini",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\pyproject.toml",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.gitignore",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_eval_regression.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_gt_loader.py",

```



```

"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_fusion_golden_telemetry.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_fusion_compare.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_served_gate_a_regression.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_per_user_regression.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_wasb_runner.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\fusion_golden.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\fusion_compare.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\gt_loader.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\regression.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\per_user_regression.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\tracknet_runner.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\eval\\baselines\\heuristic_lovo_n6.json",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\eval\\baselines\\experiment_leaderboard.json",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\CODE_MAP.md"
],
"risksGaps": [
  "RECOMMENDED PLACEMENT for new auto-discovered per-video fixtures: create
  tests/fixtures/golden/ holding small committed *_golden_features.json (one per synthetic video,
  matching the fusion_golden output schema), and a new tests/test_golden_regression.py that does
  `FIXTURE_DIR = Path(__file__).parent/'fixtures/'golden'` then
  `@pytest.mark.parametrize('fixture', sorted(FIXTURE_DIR.glob('*_golden_features.json'))
  ids=lambda p: p.stem)`. This mirrors fusion_compare.load_videos (glob+json.load) but reads
  COMMITTED files instead of tmp_path, and mirrors test_served_gate_a_regression.py's per-video
  loop ? except it runs in CI because the fixtures are committed. Resolve paths via
  Path(__file__).parent, NEVER cwd (agent cwd resets between bash calls; tests may run from any
  dir).",
  "There is no conftest.py today. If the parametrized test needs a shared loader/fixture,
  add tests/conftest.py (new file) with a small `load_golden_fixture` helper or a `golden_videos`
  fixture ? but a module-local glob is simpler and matches the existing zero-conf test convention;
  prefer that unless multiple test files need it.",
  "Do NOT reuse output/ for committed fixtures ? it is gitignored and won't survive a clean
  checkout or exist in CI (that is exactly why test_served_gate_a_regression.py skips in CI). The
  fixtures must be inside tests/ to be tracked and discovered.",
  "Privacy/licensing risk if real footage-derived data is committed: the real 6-video corpus
  is owner footage; commit only SYNTHETIC or fully-anonymized feature JSON (no frames, no minors),
  consistent with the inline-synthetic data already used in test_fusion_compare.py. If anchoring
  on real numbers, store only the derived F1/baseline in eval_baselines/ (already the convention),
  not raw player data.",
  "A committed-fixture regression test should assert against a committed expected value
  (e.g. an eval_baselines/ entry or an inline EXPECTED constant) using the existing
  regression.regress / experiment.compare / metrics.score transforms ? reuse those, do not write
  new scoring math (CODE_MAP warns calibrate_wasb.load_golden_gts + experiment harness are widely-
  shared APIs).",
  "Marker hygiene: no custom pytest markers are registered (no [tool.pytest] markers
  config). If the new test introduces a marker (e.g. @pytest.mark.golden), register it in
  pytest.ini's `markers =` or it will emit PytestUnknownMarkWarning; simpler to rely on
  parametrize + skipif like the rest of the suite."
]
},
{
  "area": "Repo constraints for committing structured regression fixtures (file-placement,
  gitignore, git-lfs, schema/version conventions, licensing/privacy guardrails, size budget)",
  "summary": "The repo deliberately commits NO raw/derived video data today: there are zero
  tracked .csv/.jsonl/fixture files, tests/ has no data dirs (tests build synthetic data in-code),
  and the ONLY committed structured data is tiny aggregate-metric JSON in eval_baselines/ (e.g.
  heuristic_lovo_n6.json ? mean_f1 + per-video scores, no per-frame data). The file-placement

```

table (CODE_MAP \$0) routes pipeline outputs (trajectories, features, DBs, frames) to gitignored output/, and a dedicated doc ? docs/GOLDEN_DATA_SHARING.md (#175) ? explicitly states the golden corpus is PROPRIETARY and its derived features must live in a PRIVATE OneDrive folder OUTSIDE the repo, never committed, with a guardrail forbidding symlinking it into a tracked path. git-lfs is INSTALLED (3.5.1, global filters) but NOT in use (no .gitattributes anywhere). On the licensing/privacy question: committing detector-derived numeric streams (shuttle X/Y, segment intervals, person/audio features) is defensible because the golden videos are owned/licensed by the founder (self-recorded + ShuttleSet, never end-user uploads) and the streams are abstract numbers, not frames ? BUT the repo's OWN convention currently says keep them out of git (private OneDrive), and minors must be excluded, so any decision to commit even derived data is a product/privacy call that is owner-gated, not a free engineering choice.",

"keyFindings": [

"(1) WHERE FIXTURES BELONG: Per docs/CODE_MAP.md \$0 file-placement table ? tests go in tests/test_*.py (pytest auto-discover, 70% coverage floor, must be pure/offline/mocked, no GPU/WSL/network). Small TRACKED metric baselines go in eval_baselines/ ('Tracked ? survives a clean checkout; the no-regression gate reads it'). Model-artifact MANIFESTS go in artifacts/<name>/<ver>/ (manifest committed, weights gitignored ? ADR-008). Pipeline run outputs (reels, TRAJECTORIES, DBs, frames) go in output/ = GITIGNORED.",

"(2) WHAT IS GITIGNORED: .gitignore ignores output/, output_dir/, scratch/, .env/, all *.db/*.db-journal/*.db-wal, *.pt (weights), .env*, coverage, __pycache__, and specific backend/static highlight mp4s. NOTE: artifacts/ itself is NOT in .gitignore (it currently shows as untracked in git status) ? only model WEIGHTS inside it are meant to be gitignored per ADR-008, the manifest is committed. scratch/ is fully gitignored+excluded from ruff/pytest.",

"(3) GIT-LFS: git-lfs v3.5.1 is installed and its filters are registered globally (filter.lfs.clean/smudge/process, required=true), BUT there is NO .gitattributes file anywhere in the repo ? so LFS tracks nothing and is effectively NOT in use. LFS is AVAILABLE if needed but the repo has chosen the 'keep big data out of git' path (output/ gitignored + private OneDrive sync) instead of LFS.",

"(4) NO EXISTING DATA FIXTURE PRECEDENT: `git ls-files` finds ZERO committed .csv/.jsonl files and no tests/ data/fixture/golden directories. Tests (e.g. test_fusion_golden_telemetry.py) build manifests + synthetic data inline in code with monkeypatch/tmp_path. The ONLY committed structured-data precedent is eval_baselines/*.json ? and those are tiny AGGREGATE metrics (mean_f1, std, per_video score dict), NOT raw per-frame or per-window derived streams.",

"(5) PROPRIETARY-DATA GUARDRAIL IS EXPLICIT: docs/GOLDEN_DATA_SHARING.md (status 2026-06-15, #175) says the GPU-extracted golden artifacts (trajectory CSVs, *_golden_features.json, manifests) sync via a PRIVATE OneDrive folder at %USERPROFILE%\OneDrive\rally-annotator\golden-data\ ? OUTSIDE the repo ? precisely so large/proprietary data is 'never committed to git.' Guardrail: 'features derive from the Proprietary corpus ? keep the OneDrive folder private (no public share links)' and 'do not symlink the shared folder into a tracked path.'",

"(6) SCHEMA/VERSION CONVENTIONS TO MIRROR: DB uses a PRAGMA user_version migration ladder in backend/infrastructure/database.py (currently at v5; ordered `if version < N:` blocks, additive ALTERs that preserve prior rows, tests assert the expected version so drift fails CI). Detector data is DETECTOR-AGNOSTIC: common fields are columns, detector-specific metrics go in a `stats` JSON blob keyed by a `detector` string ? a new detector reuses the table. eval_baselines JSON carries provenance fields (name, description, `computed` repro command, metric values). Golden features JSON has a stable shape: {name, fps, frame_width, candidates:[{start,end,shuttle{5 feats},person{5 feats},label}], gts:[[a,b]]}. Any fixture should likewise carry a version/schema tag + provenance and follow the column-vs-stats-JSON split.",

"(7) VIDEO PROVENANCE IS CLEAN-BY-DESIGN: docs/GOLDEN_SET_VENDORING.md ? 'Golden sets are built only from footage we license or own ? starting with the founder's personal collection, plus ShuttleSet, plus purpose-recorded clips. No end-user uploads ever flow to a vendor.' So the source is owned/self-recorded + ShuttleSet (broadcast contrast only), NOT third-party broadcast scraping.",

"(8) MINORS / PRIVACY GUARDRAIL: CLAUDE.md + COMMERCIALIZATION.md C14 + OWNED_MODEL_IMPLEMENTATION_PLAN mandate 'exclude minors' and 'per-user training data needs a separate explicit opt-in.' FIELD_VISIT_PLAYBOOK: 'sessions with minors are not recorded for our library ? adult sessions only.' KHELSUTRA_PRODUCT_OVERVIEW: 'Footage involving minors is never used to improve the tool and is never published.' Any committed fixture must be guaranteed minor-free at source.",

"(9) LICENSING GUARDRAIL (for completeness): CLAUDE.md ? paid LLMs (Gemini/OpenAI/Anthropic) are inference/serving ONLY, NEVER a training data source; all code+data+weights must be MIT/Apache-2.0/BSD. Derived numeric streams from owned video do not

implicate paid-LLM-output rules (they come from MIT-licensed WASB/TrackNet detectors, not from Gemini)."

```
],
"dataStructures": [
  {
    "name": "Trajectory CSV (per-video shuttle stream)",
    "where": "Produced to gitignored output/ (predict writes <stem>_predict.csv); parsed by
backend/pipeline/detectors/tracknet_runner.py::parse_trajectory_csv",
    "shapeOrFormat": "CSV header `Frame,Visibility,X,Y` ? exactly one row per video frame
(int frame index, 0/1 visibility, pixel X, pixel Y). Pure numeric, no imagery."
  },
  {
    "name": "*_golden_features.json (per-video derived feature record)",
    "where": "backend/eval/fusion_golden.py::extract_video return value; written one JSON
per video to out_dir (output/ or the private OneDrive features/ folder)",
    "shapeOrFormat": "{name, fps, frame_width, candidates:[{start, end, shuttle:{~5
fps/resolution-invariant feats}, person:{~5 person-cue feats}, label:int}],
gts:[{start,end},...]]. The replayable CPU re-scoring substrate; small (a few KB?tens of KB).",
  },
  {
    "name": "eval_baselines/*.json (the ONLY committed structured-data precedent)",
    "where": "eval_baselines/heuristic_lovo_n6.json, gemini_wrapper_2026-06-10.json,
experiment_leaderboard.json ? TRACKED in git",
    "shapeOrFormat": "Tiny aggregate-metric JSON: {name, description, computed:<repro
command>, mean_f1, std, per_video:{video_name: f1, ...}}. Carries provenance; NO per-frame/per-
window data."
  },
  {
    "name": "candidate_segments table (detector-agnostic DB schema)",
    "where": "backend/infrastructure/database.py, migration v1 (PRAGMA user_version=1)",
    "shapeOrFormat": "Columns: id, video_id, detector(str), chunk_index, start_time,
end_time, duration, confidence, stats(JSON for detector-specific metrics),
detection_params(JSON), confirmed_segment_id, human_label, notes, user_id(v4, nullable). Common
fields=columns, detector-specific=stats JSON ? the reuse pattern a fixture should mirror."
  },
  {
    "name": "PRAGMA user_version migration ladder",
    "where": "backend/infrastructure/database.py::_init_db",
    "shapeOrFormat": "Ordered `if version < N:` blocks; currently at v5 (v1
candidate_segments, v2 jobs, v3 self-scheduling, v4 user_id columns, v5 user_models). Additive
ALTERS preserve rows; tests assert expected version (drift fails CI).",
  }
],
"videoDependency": "High but CLEAN. The derived streams (shuttle X/Y trajectories,
candidate-window features, person/audio cues) are extracted FROM the golden videos, so fixtures
encode information about proprietary footage. However: (a) the videos are owned/licensed by the
founder (self-recorded + ShuttleSet), never third-party broadcast scraping and never end-user
uploads (GOLDEN_SET_VENDORING.md); (b) the fixtures are abstract NUMERIC streams (frame index,
pixel coords, scalar features) ? NOT frames/imagery, so they carry no biometric/identifiable
content; (c) the detectors that produce them (WASB/TrackNet) are MIT-licensed and the streams
contain no paid-LLM output, so the licensing guardrail is not implicated. The binding constraint
is the repo's OWN convention (GOLDEN_DATA_SHARING.md) that these proprietary-derived artifacts
live in private OneDrive and are NOT committed ? committing them is a product/privacy reversal
that is owner-gated.",
"builtVsPlanned": "BUILT: the gitignore rules; eval_baselines/ as the tracked-baseline
pattern; the PRAGMA user_version ladder (v5) with detector-agnostic stats-JSON tables;
backend/eval/fusion_golden.py producing the *_golden_features.json streams; the trajectory CSV
parse path; the test suite's synthetic-data-in-code pattern (no committed data fixtures).
CONVENTION/PLAN (not code-enforced): GOLDEN_DATA_SHARING.md private-OneDrive sync (#175,
status='convention + plan 2026-06-15' ? the optional scripts/sync_golden.py and RALLY_GOLDEN_DIR
env var are NOT required/built yet); the hard train/eval-leakage guardrails
(permissions/approval at ingest+export) are explicitly 'TODO ? design with the user' in
GOLDEN_SET_PHASE1_VIDEOS $5 / BACKLOG; the consent ledger (contribution_consent, minors-
exclusion at the writer) is PERSONALIZATION_PLAN v6 'pending.' git-lfs is available but unused
(no .gitattributes).",
```

```

"constraints": [
  "Committed fixtures = tests/test_*.py (pure/offline/mock, 70% cov floor) for logic, and
  eval_baselines/*.json for tracked metric baselines. Bulk derived data does NOT go in git ?
  output/ is gitignored and the golden artifacts are designed to live in private OneDrive, NOT the
  repo.",
  "Do NOT commit raw frames/video, *.db, *.pt weights, or large per-frame streams ? all
  gitignored by convention. Model artifacts: commit the manifest in artifacts/<name>/<ver>/,
  gitignore the weights (ADR-008).",
  "Mirror the detector-agnostic schema: common fields as columns/keys + a `stats`/params
  JSON blob keyed by a `detector` string; carry a version/schema tag + provenance (like
  eval_baselines' `computed` repro command and like PRAGMA user_version).",
  "Licensing: every dependency (code+data+weights) MIT/Apache-2.0/BSD only; paid LLMs are
  serving-only and NEVER a training/data source ? derived streams must come from the MIT
  WASB/TrackNet detectors, not Gemini output.",
  "Privacy: exclude minors at the source; per-user/contributed data needs explicit separate
  opt-in + consent ledger; footage involving minors is never used to improve the tool. Committing
  derived data from such footage would violate this even though it's 'just numbers.'",
  "Branching safety (CLAUDE.md): branch from origin/master explicitly, re-verify HEAD before
  commit, stage explicit paths (never git add -A) ? relevant when actually landing any fixture PR.
  NOTE: `git pull --ff-only` failed at session start ('Cannot fast-forward to multiple branches')
  ? the local checkout may lag origin; treat as possibly stale."
],
"relevantPaths": [
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\CODE_MAP.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_DATA_SHARING.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_VENDORING.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_PHASE1_VIDEOS.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.gitignore",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\infrastructure\\database.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\fusion_golden.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\tracknet_runner.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\eval_baselines\\heuristic_lovo_n6.json",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\COMMERCIALIZATION.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\OWNED_MODEL_IMPLEMENTATION_PLAN.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\CLAUDE.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_fusion_golden_telemetry.py"
],
"risksGaps": [
  "RECOMMENDED DESIGN TO STAY CLEAN: commit only (a) tiny aggregate/metric fixtures and
  small DERIVED NUMERIC streams (trajectory CSV rows, feature JSON) ? never frames/imagery; (b)
  from videos guaranteed owned/licensed AND minor-free at source; (c) with an explicit
  provenance/license/consent tag + a version/schema tag in each fixture. This keeps licensing
  clean (MIT detectors, no paid-LLM output) and privacy clean (no biometrics, no minors).",
  "POLICY CONFLICT TO RESOLVE WITH OWNER: GOLDEN_DATA_SHARING.md currently says the
  proprietary-derived golden features stay in PRIVATE OneDrive and are NOT committed. A 'commit a
  small regression fixture to git' design REVERSES that convention. Because the corpus is labeled
  Proprietary, this is a product/privacy decision (owner-gated), not a unilateral engineering
  choice ? get explicit sign-off and ideally commit a redacted/synthetic or a tiny owner-cleared
  subset rather than the full corpus.",
  "SIZE BUDGET: corpus is n=6 owned golden videos today
  (eval_baselines/heuristic_lovo_n6.json + LOVO docs). Per-video trajectory CSV = one row per
  frame of `Frame,Visibility,X,Y` (~20-25 bytes/row); a ~2-4 min clip at 30-60fps ? 3.6k-14.4k
  rows ? ~80KB-350KB per video ? full n=6 trajectory set ? <2MB. The *_golden_features.json (one
  record per candidate window, ~10 feats) is far smaller ? a few KB to tens of KB per video. So a
  numeric-only regression fixture for all 6 videos is comfortably git-sized (low single-digit MB);
  git-lfs/.gitattributes is NOT needed at this scale.",
  "GAP: no code-enforced guardrail yet prevents derived data from a minors/unconsented or
  unlicensed video being committed ? the leakage/consent guards are TODO (PHASE1_VIDEOS $5,

```

PERSONALIZATION_PLAN v6 pending). Until those exist, the minor-free/owned guarantee rests on manual discipline; a committed fixture should carry an explicit per-video provenance+consent assertion.",

"GAP: artifacts/ is untracked and NOT listed in .gitignore ? if a fixture-generating PR also touches artifacts/, be careful not to accidentally stage weights; stage explicit paths only.",

"STALENESS FLAG: `git pull --ff-only` failed ('Cannot fast-forward to multiple branches') ? local master may lag origin/master; re-fetch and branch from origin/master before landing any fixture PR (per CLAUDE.md shared-checkout safety)."

```
]
}
```

2. assistant (2026-06-16T02:36:09.506Z)

I'll review this design for gaps. Let me first verify key claims against the actual codebase before forming my assessment.

3. user (2026-06-16T02:36:13.023Z)

Already up to date.

---STATUS---

?? artifacts/

?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md

---BRANCH---

master

4. user (2026-06-16T02:36:13.031Z)

```
1      name: CI
2
3      on:
4        push:
5          branches: [master]
6        pull_request:
7
8      permissions:
9        contents: read
10
11     concurrency:
12       group: ci-${{ github.ref }}
13       cancel-in-progress: true
14
15     jobs:
16       # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17       # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18       # tracked .py, including scripts/ and training/, which the backend-only
19       # py_compile smoke sweep does not reach.
20       lint:
21         name: Ruff lint
22         runs-on: ubuntu-latest
23         steps:
24           - uses: actions/checkout@v6
25           - uses: actions/setup-python@v6
26             with:
27               python-version: "3.13"
28           - name: Install ruff
29             run: python -m pip install ruff
30           - name: Ruff check
31             run: ruff check . --output-format github
32
33       # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34       # per-module quarantine list ? ratchet by removing entries as modules clean
35       # up. Typed core deps are installed so mypy sees their real signatures
```

```

36     # (without them ignore_missing_imports would Any-stub half the checks).
37     typecheck:
38         name: Mypy (lenient baseline)
39         runs-on: ubuntu-latest
40         steps:
41             - uses: actions/checkout@v6
42             - uses: actions/setup-python@v6
43               with:
44                 python-version: "3.13"
45             - name: Install mypy + typed core deps
46               run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
47             - name: Mypy
48               run: mypy backend training
49
50     # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
51     # module-level import error shipping while the suite stays green.
52     # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
53     # on machines without the GPU stack (test_import_smoke stubs what's absent).
54     import-smoke:
55         name: Syntax sweep + import smoke (no GPU stack)
56         runs-on: ubuntu-latest
57         steps:
58             - uses: actions/checkout@v6
59             - uses: actions/setup-python@v6
60               with:
61                 python-version: "3.13"
62             - name: Install minimal deps
63               run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
64             - name: Run import smoke tests
65               run: python -m pytest tests/test_import_smoke.py -v
66
67     full-suite:
68         name: Full test suite (offline, mocked)
69         runs-on: ubuntu-latest
70         steps:
71             - uses: actions/checkout@v6
72             - uses: actions/setup-python@v6
73               with:
74                 python-version: "3.13"
75                 cache: pip
76             - name: System libs for opencv-python
77               run: |
78                 sudo apt-get update
79                 sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
80             - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
81               # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
82               # need an out-of-band index that PEP 621 can't express. Keep installing
83               # them separately here, before the editable install pulls the rest.
84               run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
85             - name: Install package (editable) + dev tooling
86               # Editable PEP 621 install: runtime deps + the `dev` group
87               # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
88               run: python -m pip install -e .[dev]
89             # obsreport (private, soft dep) is absent here: reporting tests skip via
90             # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
91             # is therefore calibrated against THIS lane's number (local runs with
92             # obsreport installed measure a little higher).
93             # Floor calibrated 2026-06-11 against this lane's measured 72% (local
94             # with obsreport: 73%). Ratchet it UP as coverage grows; never down
95             # without an owner decision.
96             - name: Run full suite (with coverage floor)

```

```
97         run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
98
```

5. user (2026-06-16T02:36:13.032Z)

```
1     """Regression runner ? verify the production tool against golden-set ground truth.
2
3     Implements the explicit ask: *"given a video, get a golden set created and verify
4     its precision and recall against the production tool."* This is deliberately a thin
5     wrapper over machinery that already exists:
6
7     ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8     predictions  ? the production pipeline's stored DB rows    [backend.eval.harness]
9     comparison   ? interval matching + P/R/F1                  [backend.eval.metrics]
10    baseline     ? a per-video snapshot of "known good" numbers [this module]
11
12    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16    """
17    import os
18    import json
19    import hashlib
20    import logging
21    from dataclasses import dataclass, asdict
22    from typing import Dict, List, Optional
23
24    from backend.infrastructure.database import Database
25    from backend.eval import metrics
26    from backend.eval.harness import get_predictions
27    from backend.annotations import annotation_registry
28
29    logger = logging.getLogger(__name__)
30
31    # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32    # actually load a committed baseline to compare against.
33    DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34    DEFAULT_TOLERANCE = 0.05
35
36
37    def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38        """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
39        norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40        return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43    @dataclass
44    class RegressionResult:
45        video_id: str
46        stage: str
47        iou_threshold: float
48        precision: float
49        recall: float
50        f1: float
51        # Baseline values compared against (None when no baseline existed yet).
52        baseline_precision: Optional[float]
53        baseline_recall: Optional[float]
54        tolerance: float
55        regressed: bool
56        reasons: List[str]
57        # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58        # from "nothing to compare" so the CLI can refuse to silently green-light the
```

```

latter.
59     has_baseline: bool = False
60     gt_hash: Optional[str] = None
61
62     def as_dict(self) -> dict:
63         return asdict(self)
64
65
66     # ----- baseline store
67
68     def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69         """Load the per-video baseline snapshot file (returns {} if absent)."""
70         if not os.path.isfile(path):
71             return {}
72         with open(path) as f:
73             return json.load(f)
74
75
76     def _baseline_key(video_id: str, stage: str) -> str:
77         return f"{video_id}::{stage}"
78
79
80     def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                      f1: float, path: str = DEFAULT_BASELINE_PATH,
82                      iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                      gt_fingerprint: Optional[str] = None) -> None:
84         """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86         Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87         later run computed under different settings (or against different labels) is
88         as incommensurable rather than silently compared.
89         """
90         baselines = load_baselines(path)
91         baselines[_baseline_key(video_id, stage)] = {
92             "video_id": video_id, "stage": stage,
93             "precision": precision, "recall": recall, "f1": f1,
94             "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95         }
96         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97         with open(path, "w") as f:
98             json.dump(baselines, f, indent=2, sort_keys=True)
99         logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
100                    _baseline_key(video_id, stage), precision, recall)
101
102     # ----- the runner
103
104     def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105               stage: str = "final", iou_threshold: float = 0.5,
106               detector: Optional[str] = None,
107               tolerance: float = DEFAULT_TOLERANCE,
108               baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109         """Score stored predictions for one video against ground truth and compare to
110         baseline.
111
112         `ground_truth` is a list of (start, end) intervals ? typically obtained from an
113         AnnotationProvider (see `regress_with_provider`). A regression is flagged when
114         precision OR recall falls more than `tolerance` below the snapshotted baseline.
115         If no baseline exists yet, `regressed` is False (nothing to compare to) and the
116         caller can choose to snapshot the current numbers via `save_baseline`.
117         """
118         preds = get_predictions(db, video_id, stage, detector=detector)
119         s = metrics.score(preds, ground_truth, iou_threshold)
120         fp = gt_hash(ground_truth)

```



```

120
121     baselines = load_baselines(baseline_path)
122     b = baselines.get(_baseline_key(video_id, stage))
123
124     reasons: List[str] = []
125     regressed = False
126     has_baseline = b is not None
127     base_p = base_r = None
128     if b is not None:
129         base_p, base_r = b["precision"], b["recall"]
130         # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
131         # against different labels is not a valid reference ? flag it instead of
trusting it.
132         b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
133         if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
134             regressed = True
135             reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
136         if b_det is not None and b_det != detector:
137             regressed = True
138             reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139         if b_hash is not None and b_hash != fp:
140             regressed = True
141             reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142         if s.precision < base_p - tolerance:
143             regressed = True
144             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145         if s.recall < base_r - tolerance:
146             regressed = True
147             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148     else:
149         reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150
151     return RegressionResult(
152         video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153         precision=s.precision, recall=s.recall, f1=s.f1,
154         baseline_precision=base_p, baseline_recall=base_r,
155         tolerance=tolerance, regressed=regressed, reasons=reasons,
156         has_baseline=has_baseline, gt_hash=fp,
157     )
158
159
160 def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                          tier: str = "file", stage: str = "final",
162                          iou_threshold: float = 0.5,
163                          detector: Optional[str] = None,
164                          tolerance: float = DEFAULT_TOLERANCE,
165                          baseline_path: str = DEFAULT_BASELINE_PATH,
166                          guideline: Optional[str] = None,
167                          sport: str = "badminton",
168                          provider_kwargs: Optional[dict] = None) -> RegressionResult:
169     """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170
171     `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172     once those providers land). `video_ref` is whatever that provider expects (a CSV
173     path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174     predictions in the DB (the file MD5).
175     """

```

```

176         provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177         gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178         return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                        detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185                f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
188                    tol={r.tolerance}"
189             return head + "".join(f"\n    - {why}" for why in r.reasons)

```

6. assistant (2026-06-16T02:36:16.610Z)

Let me verify more claims: the experiment/windowing/distill_local APIs the design leans on, and the test templates.

7. user (2026-06-16T02:36:17.953Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. No new scoring math lives
7     here*; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23
24    Three modes (`EXPERIMENT_HARNESS.md` §5):
25    - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26      aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27      held-out loop, but variant-parameterised.
28    - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29      variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30      human spot-checks (and what two highlight reels would show side by side).
31    - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32      (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34    Pure + offline + unit-tested. The only DB-touching helper is
35    `load_video_candidates`, which reads persisted candidates via
36    `Database.query_candidate_segments`.
37    """
38    from __future__ import annotations
39
40    import json
41    import logging
42    import os

```

```

43     from dataclasses import dataclass, field
44     from datetime import datetime, timezone
45     from hashlib import sha1
46     from typing import Any, Callable, Dict, List, Optional, Sequence
47
48     from backend.eval.calibration import pct_improvement
49     from backend.eval.classifier import LogisticRegression
50     from backend.eval.gemini_refine import merge_overlaps
51     from backend.eval.metrics import Interval, match_intervals, score
52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
ratio
56     from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
scores
57
58     logger = logging.getLogger(__name__)
59
60     # Where a comparison run appends one reproducible record. Tracked location (NOT under
61     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62     # regression.DEFAULT_BASELINE_PATH.
63     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64     _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67     class ExperimentError(ValueError):
68         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69         score an unlabeled video with no pinned model, or a gate referencing an absent
70         feature)."""
71
72
73     # ----- Variant
74
75
76     @dataclass
77     class Variant:
78         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80         A variant turns one video's persisted candidate windows + features into rally
81         intervals. Every knob defaults to the **control** behaviour (no gate, no learned
82         filter), so a variant that merely *carries* a new, disabled cue is byte-identical
83         to control ? the core no-regression guarantee.
84
85         Fields:
86         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87           for the leaderboard and for selecting the served path (the existing
88           ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
89           New cues are recorded here default-OFF.
90         - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
91           ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92           ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93           windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94         - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95           whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96           cue persists that feature).
97         - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98           ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99           required for ``compare_unlabeled`` on a learned variant.
100        - ``feature_names`` ? the persisted features the learned filter consumes. When set
101          WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102          (honest) ? the recipe, not a pinned artifact.
103        - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104          overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.

```

```

105     """
106
107     name: str
108     segmenter: str = "wasb_hybrid"
109     config_overrides: Dict[str, Any] = field(default_factory=dict)
110     cue_flags: Dict[str, bool] = field(default_factory=dict)
111     min_crossings: int = 0
112     min_players: int = 0
113     classifier_model: Optional[str] = None
114     feature_names: Optional[List[str]] = None
115     threshold: float = 0.5
116     merge_gap: float = 1.0
117     # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118     # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119     # failure where a fixed 0.5 zeroed out a small-candidate video.
120     tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121     calibrate: Optional[str] = None        # None | "platt" | "isotonic" ? calibrate
proba first
122
123     def is_learned(self) -> bool:
124         """True if this variant applies a learned classifier (pinned model or
recipe)."""
125         return self.classifier_model is not None or bool(self.feature_names)
126
127     def to_dict(self) -> dict:
128         return {
129             "name": self.name,
130             "segmenter": self.segmenter,
131             "config_overrides": self.config_overrides,
132             "cue_flags": self.cue_flags,
133             "min_crossings": self.min_crossings,
134             "min_players": self.min_players,
135             "classifier_model": self.classifier_model,
136             "feature_names": self.feature_names,
137             "threshold": self.threshold,
138             "merge_gap": self.merge_gap,
139             "tune_threshold": self.tune_threshold,
140             "calibrate": self.calibrate,
141         }
142
143     @classmethod
144     def from_dict(cls, d: dict) -> "Variant":
145         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146         return cls(**{k: v for k, v in d.items() if k in known})
147
148     def spec_hash(self) -> str:
149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
the
151         same, so a regression is always traceable to a recipe change, never to drift."""
152         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153         return sha1(blob.encode()).hexdigest()[:12]
154
155
156     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157     #: filter. Always the comparison reference; "rollback" = make this the served variant.
158     CONTROL = Variant(name="control")
159
160
161     # ----- persisted candidates
162
163
164     @dataclass

```

```

165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:
186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                        name, vc.name)
188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
193
194
195 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196     cols = [_feature_column(vc, name) for name in feature_names]
197     return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200 def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201     """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202     players)."""
203     n = len(vc.intervals)
204     mask = [True] * n
205     if variant.min_crossings > 0:
206         col = _feature_column(vc, "net_crossings")
207         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208     if variant.min_players > 0:
209         col = _feature_column(vc, "players_in_court")
210         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211     return mask
212
213
214 def predict(variant: Variant, vc: VideoCandidates,
215             model: Optional[LogisticRegression] = None,
216             threshold: Optional[float] = None,
217             calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
218     """Variant prediction: gate by persisted features, optionally filter by a learned
219     model, then merge. Pure and deterministic.
220
221     ``model`` (an already-fitted ``LogisticRegression``) is supplied by ``compare`` per LOVO
222     fold; if omitted and the variant pins ``classifier_model``, that frozen model is
223     loaded. A learned variant with neither a supplied nor a pinned model raises ? there
224     is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
225     fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
226     them.
227
228     """
229     mask = _gate_mask(variant, vc)

```

```

227     if variant.feature_names:
228         if model is None:
229             if variant.classifier_model is None:
230                 raise ExperimentError(
231                     f"variant {variant.name!r} is learned but has no model to predict "
232                     f"with (pass a fitted model or set classifier_model)")
233                 model = LogisticRegression.load(variant.classifier_model)
234                 proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
235                 if calibrator is not None:
236                     proba = [calibrator(p) for p in proba]
237                 thr = variant.threshold if threshold is None else threshold
238                 mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
239                 kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
240                 return merge_overlaps(kept, gap_tol=variant.merge_gap)
241
242
243     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
244                             train_videos: Sequence[VideoCandidates], iou: float
245                             ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
246         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
247         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
248         (use the variant default). Honest: never looks at the held-out video.
249         """
250         calibrator: Optional[Callable[[float], float]] = None
251         if variant.calibrate:
252             raw: List[float] = []
253             y: List[int] = []
254             for vc in train_videos:
255                 raw.extend(float(p) for p in
256                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
257                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
258             if variant.calibrate == "platt":
259                 calibrator = fit_platt(raw, y)
260             elif variant.calibrate == "isotonic":
261                 calibrator = fit_isotonic(raw, y)
262             else:
263                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
264             threshold: Optional[float] = None
265             if variant.tune_threshold:
266                 best_t, best_f1 = variant.threshold, -1.0
267                 for k in range(1, 20):
268                     t = round(0.05 * k, 2)
269                     f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
270                                 vc.gts or [], iou).f1 for vc in train_videos]
271                     mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272                     if mean_f1 > best_f1:
273                         best_f1, best_t = mean_f1, t
274                 threshold = best_t
275             return calibrator, threshold
276
277
278     # ----- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """

```

```

284     X: List[List[float]] = []
285     y: List[int] = []
286     for vc in videos:
287         if vc.gts is None:
288             raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))
291     if not X:
292         raise ExperimentError("cannot train a learned variant on zero candidates")
293     return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296 def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                     iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298     """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299     mean aggregate.
300
301     Prediction policy:
302     - **static variant** (no learned filter): predict each video directly ? no
303     training,
304     so no leakage; per-video scoring is already honest.
305     - **learned, pinned model** (`classifier_model` set): score every video with the
306     SHIPPED model. ? optimistic if those videos were in its training set ? use this
307     to
308     report "how the shipped artifact does here", not for the promotion decision.
309     - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
310     ?
311     train on the OTHER videos, predict the held-out one. The number to trust.
312     - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
313     only).
314     """
315     for vc in videos:
316         if vc.gts is None:
317             raise ExperimentError(f"{vc.name} has no golden labels; use
318             compare_unlabeled")
319
320     per_video: List[Dict[str, Any]] = []
321     predictions: Dict[str, List[Interval]] = {}
322
323     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
324     pinned_model = (LogisticRegression.load(variant.classifier_model)
325                     if variant.classifier_model is not None else None)
326     all_model = None
327     if recipe_lovo and not honest:
328         all_model = _train(variant, videos, iou)
329
330     for i, vc in enumerate(videos):
331         cal: Optional[Callable[[float], float]] = None
332         thr: Optional[float] = None
333         if recipe_lovo and honest:
334             others = [v for j, v in enumerate(videos) if j != i]
335             if not others:
336                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
337                 pinned model")
338             model: Optional[LogisticRegression] = _train(variant, others, iou)
339             if variant.tune_threshold or variant.calibrate: # operating point from
340                 the train fold
341                 assert model is not None # _train never returns
342                 None
343                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
344             elif recipe_lovo: # honest=False ? one model over all
345                 videos
346                 model = all_model
347             else: # static variant or pinned model
348                 model = pinned_model

```

```

340         preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341         assert vc.gts is not None # guarded at function top
342         sc = score(preds, vc.gts, iou)
343         per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344         predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                     for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),
351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])
375             ratios.append(_segm.segment_count_ratio(preds, gts))
376
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {

```



```

395         "variant_a_ci": _ci_block(eval_a),
396         "variant_b_ci": _ci_block(eval_b),
397         "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398         "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                          "variant_b": _segmentation_block(videos, eval_b)},
400     }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427         per_video_delta = [
428             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429             for p in eval_b["per_video"]
430         ]
431
432         # One fingerprint over the whole label set ? detects "the deltas were measured
against
433         # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434         all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436         result: Dict[str, Any] = {
437             "iou": iou,
438             "label_set_hash": gt_hash(all_gts),
439             "num_videos": len(videos),
440             "variant_a": eval_a,
441             "variant_b": eval_b,
442             "delta": delta,
443             "delta_pct": delta_pct,
444             "per_video_delta": per_video_delta,
445         }
446         if honest_stats:
447             result["honest"] = honest_summary(videos, eval_a, eval_b)

```

```

448         return result
449
450
451 # ----- SxS on unlabeled video
452
453
454 def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455                      agree_iou: float = 0.5) -> Dict[str, Any]:
456     """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458     With no labels there is no lift to measure ? instead this surfaces where the two
459     variants **agree vs diverge**, which is exactly what a human reviews (and what two
460     highlight reels would show side by side):
461
462     - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463     - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464     - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPS:
465       the human / a later golden label adjudicates).
466
467     Both variants must be predictable without labels: a learned variant therefore needs
468     a
469     pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470     ``metrics.match_intervals`` ? no new matching logic.
471     """
472     preds_a = predict(variant_a, video)
473     preds_b = predict(variant_b, video)
474     mr = match_intervals(preds_a, preds_b, agree_iou)
475
476     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
477              for m in mr.matches]
478     agree_ious = [m.iou for m in mr.matches]
479     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
480     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
481
482     def _dur(ivs: List[Interval]) -> float:
483         return sum(e - s for s, e in ivs)
484
485     return {
486         "video": video.name,
487         "agree_iou": agree_iou,
488         "variant_a": variant_a.name,
489         "variant_b": variant_b.name,
490         "num_a": len(preds_a),
491         "num_b": len(preds_b),
492         "num_agreed": len(agreed),
493         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
494         "duration_a": _dur(preds_a),
495         "duration_b": _dur(preds_b),
496         "agreed": agreed,
497         "a_only": a_only,
498         "b_only": b_only,
499     }
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {

```

```

512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                      "spec_hash": result["variant_a"]["spec_hash"],
518                      "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                      "spec_hash": result["variant_b"]["spec_hash"],
521                      "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
the
525     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
526     honest = result.get("honest") or {}
527     if honest.get("paired_delta_f1"):
528         row["honest_delta_f1"] = honest["paired_delta_f1"]
529     entries: List[Dict[str, Any]] = []
530     if os.path.isfile(path):
531         with open(path, encoding="utf-8") as f:
532             entries = json.load(f)
533     entries.append(row)
534     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
535     with open(path, "w", encoding="utf-8") as f:
536         json.dump(entries, f, indent=2)
537     logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
538               row["variant_a"]["name"], row["variant_b"]["name"],
539               (result.get("delta") or {}).get("f1", 0.0), path)
540     return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                             gts: Optional[List[Interval]] = None,
545                             feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
547         `Database.query_candidate_segments`.
548         ``feature_names`` selects which persisted features to lift out of each row's
549         ``stats``
550         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
551         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
552         but works against any `Database` (including an in-memory one in tests).
553         """
554         rows = db.query_candidate_segments(video_id, detector=detector)
555         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
556         features: Dict[str, List[float]] = {}
557         for name in feature_names:
558             if name == "duration":
559                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
560             else:
561                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
562         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)

```

8. user (2026-06-16T02:36:18.171Z)

```

1     """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
footage.

```

```

2
3     The flagged gap (#146): the served default flip (`default_segmenter=fusion_hybrid` +
4     ``min_crossings=2``) is **mock-tested only** on real video (`test_fusion_hybrid.py`
drives a
5     MagicMock runner). The offline ablation litmus
6     (`test_ablation.test_litmus_reproduces_gate_a`)
7     validates the Gate-A *signal* ? but on the **permissive proposal windowing**
8     (`distill_local.PROPOSAL`), NOT the production operating point. This module closes the
gap: it
9     runs the real ``FusionSegmenter`` decision seams (`enrich_candidate_stats` net-
crossings +
10    ``select_for_handoff`` Gate A) over windows from the **production**
11    ``trajectory_to_action_windows`` config, on the 6 cached golden WASB trajectories, and
asserts
12    the served path's measured behaviour.
13
14    GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
cleanly in
15    CI where the golden trajectories are absent; runs on the owner box where they are
present.
16
17    HONEST FINDING baked into the assertions (2026-06-14): on the **production windowing**
Gate A is
18    NOT the +0.074 win the offline harness reports. Production windowing is already
conservative
19    (velocity_threshold 0.02, merge_gap 2.0, min_window 2.0) ? few long windows, over-seg
ratio
20    already < 1 ? there is little held-shuttle junk left for Gate A to cut. So the served
?F1 is
21    ~neutral (slightly negative), NOT +0.074. The +0.074 reproduces ONLY under the
permissive
22    proposal windowing (asserted in `test_proposal_windowing_reproduces_offline_lift`
below ? that
23    is what the offline litmus measures). These tests pin BOTH facts so a future windowing
change
24    that silently revives ? or breaks ? either one trips the litmus.
25    """
26
27    from __future__ import annotations
28
29    import json
30    import os
31
32    import pytest
33
34    # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
without it
35    # (CI), skip the whole module rather than erroring at import.
36    pytest.importorskip("cv2")
37
38    from backend.eval import served_gate_a as sga # noqa: E402
39    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
40    from backend.pipeline.detectors import rally_gate # noqa: E402
41    from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
42    from backend.eval.metrics import score # noqa: E402
43    from backend.eval import segmentation_metrics as _segm # noqa: E402
44
45    # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
46    # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
47    # out-of-tree checkout.
48    _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
49    MANIFEST = os.environ.get(
        "SERVED_GATE_A_MANIFEST",
        os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),

```

```

50     )
51
52
53     def _golden_specs():
54         """Return the manifest specs whose trajectory + label files are all present, else
55         []."""
56         if not os.path.isfile(MANIFEST):
57             return []
58         with open(MANIFEST, encoding="utf-8") as f:
59             specs = json.load(f)
60             ok = [s for s in specs
61                   if os.path.isfile(str(s.get("trajectory", "")))
62                   and os.path.isfile(str(s.get("labels", "")))]
63             return ok if len(ok) >= 6 else []
64
65     _SPECS = _golden_specs()
66     _skip = pytest.mark.skipif(
67         not _SPECS,
68         reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
69     )
70
71
72     # ----- served-path
73     litmus
74     @_skip
75     def test_served_path_runs_on_all_six_golden_videos():
76         """The production FusionSegmenter decision path runs end-to-end (no GPU/Gemini) on
77         all 6
78         golden trajectories and yields a handoff window list for each, for both gate
79         settings."""
80         results = sga.evaluate_manifest(_SPECS)
81         assert len(results) == 6
82         for r in results:
83             assert r.num_gt > 0
84             assert r.n_handoff_off >= r.n_handoff_on # Gate A only ever drops windows,
85             never adds
86
87     @_skip
88     def test_served_gate_a_does_not_regress_f1():
89         """SERVED no-regression guard: on the production windowing Gate A must not
90         MATERIALLY hurt
91         F1. It is ~neutral here (the held-shuttle junk the offline harness sees is already
92         merged
93         away by the conservative production windowing) ? so we assert the mean ?F1 sits in a
94         tight
95         band around 0, never a real regression. (The +0.074 lift lives under the permissive
96         proposal
97         windowing ? see the dedicated test below.)"""
98         results = sga.evaluate_manifest(_SPECS)
99         agg = sga.aggregate(results)
100         # Honest measured value (2026-06-14): mean served ?F1 ? -0.008. Pin a tolerance band
101         so a
102         # real regression (Gate A actively destroying served F1) trips, while the measured
103         ~neutral
104         # result passes.
105         assert -0.03 <= agg["mean_delta_f1"] <= 0.03, agg
106         # And Gate A must never collapse served F1 to ~0 on the production path.
107         assert agg["mean_f1_on"] >= 0.5 * agg["mean_f1_off"] - 1e-9, agg
108
109     @_skip
110     def test_served_gate_a_does_not_increase_over_segmentation():

```

```

104         """Gate A only ever DROPS windows, so the served over-seg ratio must not rise when
it's on
105         (the direction-of-effect invariant ? even where the absolute lift is ~neutral)."""
106         results = sga.evaluate_manifest(_SPECS)
107         agg = sga.aggregate(results)
108         assert agg["mean_seg_ratio_on"] <= agg["mean_seg_ratio_off"] + 1e-9, agg
109         # Per-video too: the gate is monotone (kept windows ? all windows).
110         for r in results:
111             assert r.seg_ratio_on <= r.seg_ratio_off + 1e-9, r
112
113
114     @_skip
115     def test_served_uses_production_windowing_operating_point():
116         """Pin the served operating point to config.json's indexing defaults ? if someone
retunes
117         production windowing, this litmus's premise (and the divergence it documents) is
revisited
118         deliberately, not silently."""
119         assert sga.PROD_VELOCITY_THRESH == 0.02
120         assert sga.PROD_MERGE_GAP == 2.0
121         assert sga.PROD_MIN_WINDOW == 2.0
122
123
124     # ----- the +0.074 lives under PROPOSAL
windowing
125
126     # distill_local.PROPOSAL ? the permissive, recall-oriented candidate windowing the
OFFLINE
127     # ablation/fusion_compare harness builds on (low velocity thresh, short merge/min-dur ?
many
128     # small windows). The +0.074 Gate-A lift is a property of THIS windowing, not the served
one.
129     _PROPOSAL = (0.005, 1.0, 0.5)
130
131
132     @_skip
133     def test_proposal_windowing_reproduces_offline_lift():
134         """CONTROL / root-cause pin: under the OFFLINE harness's permissive proposal
windowing, the
135         SAME ``rally_gate`` net-crossing gate the served seam uses reproduces the offline
litmus
136         (?F1 ? +0.074, 6/6 wins, over-seg 1.68?1.01). This proves the served ~neutral result
is a
137         WINDOWING difference, not a broken gate ? and locks the offline number to its true
premise."""
138         d_on, d_off, seg_on, seg_off = [], [], [], []
139         for s in _SPECS:
140             fps, fw = float(s["fps"]), float(s["frame_width"])
141             pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
142             wins = TrackNetRunner.trajectory_to_action_windows(
143                 pts, fps=fps, frame_width=fw, velocity_thresh=_PROPOSAL[0],
144                 merge_gap=_PROPOSAL[1], min_window_duration=_PROPOSAL[2])
145             iv = [(w["start"], w["end"]) for w in wins]
146             est = rally_gate.estimate_play_axis(pts)
147             gated = rally_gate.gate_windows(pts, iv, fps, min_crossings=0, estimate=est)
148             on = [(g.start, g.end) for g in gated if g.crossings >= 2]
149             gts = load_golden_gts(s["labels"])
150             d_on.append(score(on, gts).f1)
151             d_off.append(score(iv, gts).f1)
152             seg_on.append(_segm.segment_count_ratio(on, gts))
153             seg_off.append(_segm.segment_count_ratio(iv, gts))
154         n = len(d_on)
155         mean_delta = (sum(d_on) - sum(d_off)) / n
156         wins = sum(1 for a, b in zip(d_on, d_off) if a > b + 1e-9)
157         assert mean_delta == pytest.approx(0.074, abs=0.01), mean_delta

```

```

158         assert wins == 6, wins
159         assert sum(seg_off) / n == pytest.approx(1.68, abs=0.05)
160         assert sum(seg_on) / n == pytest.approx(1.01, abs=0.05)
161

```

9. assistant (2026-06-16T02:36:21.676Z)

Now let me verify the windowing/distill_local APIs (presets, FEATURE_NAMES), the gt_loader strict mode, and CODE_MAP \$0 file-placement table.

10. user (2026-06-16T02:36:22.636Z)

```

1      """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3      **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
4      ?
5      ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was
6      built
7      on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
8      deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
9      pipeline
10     (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
11     merge_gap=2.0,
12     min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
13     is
14     measured on candidates **production never serves**, so a "win" need not transfer. This
15     is
16     exactly how Gate-A scored **0.074** on the proposal set yet **0.008** on the served
17     set
18     (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify/`).
19
20     **The fix.** One module owns the two named windowings as :class:`WindowingPreset`, and
21     the
22     SERVED preset is **single-sourced from the same Pydantic config defaults production
23     reads**
24     (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
25     Helpers
26     build candidates under either preset, and :func:`served_windowing_from_config` lifts the
27     *actual* served windowing out of a live config (overrides included), so an A/B can
28     target the
29     operating point a given deployment truly serves.
30
31     The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
32     windowing* ? lives in :mod:`backend.eval.promotion` (which composes `eval_stats` +
33     `per_user_gate`). This module is the **what windowing** that one is the **promotion
34     gate**.
35
36     Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
37     a
38     caller opts in (`distill_local` re-points its module-level presets here, byte-
39     identically).
40
41     """
42     from __future__ import annotations
43
44     from dataclasses import dataclass
45     from typing import Any, Dict, List, Mapping, Tuple
46
47     from backend.config.models import IndexingConfig
48
49     Interval = Tuple[float, float]
50
51     __all__ = [
52         "WindowingPreset",
53         "SERVED",
54         "PROPOSAL",

```

```

40     "PRESETS",
41     "served_windowing_from_config",
42     "build_windows",
43     "windows_to_intervals",
44 ]
45
46
47 @dataclass(frozen=True)
48 class WindowingPreset:
49     """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51     ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52     "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53     ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54     knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
55     """
56
57     name: str
58     velocity_thresh: float
59     merge_gap: float
60     min_window_duration: float
61
62     def as_tuple(self) -> Tuple[float, float, float]:
63         """(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
64         ``trajectory_to_action_windows`` / ``distill_local`` call sites already
speak."""
65         return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
66
67     def to_dict(self) -> Dict[str, Any]:
68         return {
69             "name": self.name,
70             "velocity_thresh": self.velocity_thresh,
71             "merge_gap": self.merge_gap,
72             "min_window_duration": self.min_window_duration,
73         }
74
75
76     def _served_preset_from_defaults() -> WindowingPreset:
77         """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
78
79         ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
80         - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
81         - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
82         - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
83
84         Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
85         point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
86         so the two can never silently diverge again.
87         """
88         idx = IndexingConfig() # construct with defaults ? the served operating point
89         return WindowingPreset(
90             name="served",
91             velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet
default 0.02
92             merge_gap=float(idx.dead_time_merge_gap),

```



```

93         min_window_duration=float(idx.min_action_window_duration),
94     )
95
96
97     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
98     #: only promotable if it holds up HERE ? the operating point real users get.
99     SERVED: WindowingPreset = _served_preset_from_defaults()
100
101     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
102     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
103     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
104     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
105     PROPOSAL: WindowingPreset = WindowingPreset(
106         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
107     )
108
109     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
110     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
111
112
113     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
114         """The SERVED windowing a *specific* live config serves (overrides honoured).
115
116         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
117         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
118         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
119         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
120         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
121         """
122         idx = _as_mapping(config, "indexing")
123         family_cfg = _as_mapping(idx, family)
124         return WindowingPreset(
125             name=f"served:{family}",
126             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
127             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
128             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
129         )
130
131
132     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
133         """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config
section
134         (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
135         if obj is None:
136             return {}
137         if isinstance(obj, Mapping):
138             sub = obj.get(key, {})
139         elif hasattr(obj, "get"):
140             sub = obj.get(key, {})
141         else:
142             sub = getattr(obj, key, {})
143         return sub if isinstance(sub, Mapping) else {}
144

```

```

145
146     def build_windows(points: List[Any], fps: float, frame_width: float,
147                       preset: WindowingPreset) -> List[Dict[str, Any]]:
148         """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing
call site).
149
150         A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a
named preset
151         instead of three loose floats, so eval candidate-building and the served path
reference the
152         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
153         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
154
155         vt, mg, mwd = preset.as_tuple()
156         return TrackNetRunner.trajectory_to_action_windows(
157             points, fps=fps, frame_width=frame_width,
158             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
159
160
161     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
162         """``[{start, end, ...}]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate
speak."""
163         return [(w["start"], w["end"]) for w in windows]
164

```

11. user (2026-06-16T02:36:23.068Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9         WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10         filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11         active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12         -> rally windows.
13
14     Training labels come from Gemini full-context labels via the SAME IoU matching the
15     evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16     other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17     compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18     rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20     Run:
21         python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22     """
23     from __future__ import annotations
24
25     import json
26     import argparse
27     from typing import Dict, List, Optional, Tuple
28
29
30     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31     from backend.pipeline.detectors import rally_gate
32     from backend.eval.calibrate_wasb import load_golden_gts
33     from backend.eval.training import label_candidates
34     from backend.eval.classifier import LogisticRegression
35     from backend.eval.metrics import score
36     from backend.eval.gemini_refine import merge_overlaps

```

```

37     from backend.eval import windowing as _windowing
38
39     Interval = Tuple[float, float]
40
41     FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
42     # Recall-oriented proposal: deliberately permissive so real rallies are among the
43     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
44     # Single-sourced from backend.eval.windowing so eval and serve can never drift ? see
that
45     # module's docstring for the eval?serve bug these named presets close. The bare tuples
are
46     # preserved (byte-identical: PROPOSAL == (0.005, 1.0, 0.5), BASELINE_LOCAL == SERVED ==
47     # (0.02, 2.0, 2.0)) so every existing call site / golden JSON stays valid.
48     PROPOSAL = _windowing.PROPOSAL.as_tuple()
49     BASELINE_LOCAL = _windowing.SERVED.as_tuple() # the served (production) windowing, no
learning
50
51
52     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
53                         proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
54         """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
55         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
56         vt, mg, mwd = proposal
57         wins = TrackNetRunner.trajectory_to_action_windows(
58             points, fps=fps, frame_width=frame_width,
59             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
60         intervals = [(w["start"], w["end"]) for w in wins]
61         gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
crossings
62         X: List[List[float]] = []
63         for w, g in zip(wins, gated):
64             dur = max(1e-6, w["end"] - w["start"])
65             win_frames = max(1.0, dur * fps)
66             X.append([
67                 dur, # rally length (sec) ?
68                 float(w["peak_velocity"]), # already normalized by
69                 float(w["mean_velocity"]),
70                 float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
71                 float(g.crossings), # net crossings (count) ?
72             ])
73         return intervals, X
74
75
76     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
77         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
78         vt, mg, mwd = BASELINE_LOCAL
79         wins = TrackNetRunner.trajectory_to_action_windows(
80             points, fps=fps, frame_width=frame_width,
81             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
82         return [(w["start"], w["end"]) for w in wins]
83
84
85     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
86         fps, fw = float(spec["fps"]), float(spec["frame_width"])
87         intervals, X = build_candidates(spec["trajectory"], fps, fw)
88         gts = load_golden_gts(spec["labels"])
89         y = label_candidates(intervals, gts, iou)

```

```

90         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
91                 "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}}
92
93
94     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
95                       threshold: float = 0.5) -> List[Interval]:
96         if not intervals:
97             return []
98         proba = model.predict_proba(X)
99         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
100         return merge_overlaps(pos, gap_tol=1.0)
101
102
103     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
104            model_out: Optional[str] = None) -> dict:
105         videos = [load_video(s, iou) for s in specs]
106         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
107               f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
108         for v in videos:
109             ceil = score(v["intervals"], v["gts"], iou).recall
110             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
111                   f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
112
113         result: dict = {"videos": [v["name"] for v in videos]}
114         if len(videos) >= 2:
115             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
116             clf, base = [], []
117             for i, held in enumerate(videos):
118                 others = [v for j, v in enumerate(videos) if j != i]
119                 X = [row for v in others for row in v["X"]]
120                 y = [t for v in others for t in v["y"]]
121                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
122                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
123                 sc = score(preds, held["gts"], iou)
124                 bsc = score(held["baseline"], held["gts"], iou)
125                 clf.append(sc)
126                 base.append(bsc)
127                 print(f"    held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}"
128                       f"    | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
129             def mean(xs, a):
130                 return sum(getattr(s, a) for s in xs) / len(xs)
131             print(f"    MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
R={mean(clf, 'recall'):.3f} "
132                   f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
133                   f"R={mean(base, 'recall'):.3f} F1={mean(base, 'f1'):.3f}")
134             result["mean_learned_f1"] = mean(clf, "f1")
135             result["mean_baseline_f1"] = mean(base, "f1")
136
137         # Final model on ALL videos (for production / running on a fresh held-out video).
138         Xall = [row for v in videos for row in v["X"]]
139         yall = [t for v in videos for t in v["y"]]
140         final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
141         assert final.w is not None # set by fit()
142         weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
143         print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
144         if model_out:
145             final.save(model_out)
146             print(f"final model (all videos) -> {model_out}")
147             result["model_out"] = model_out

```

```

148         return result
149
150
151     def main() -> None:
152         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
153         ap.add_argument("--manifest", required=True,
154                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
155         ap.add_argument("--iou", type=float, default=0.5)
156         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
157         ap.add_argument("--model-out", default="output/local_rally_model.json")
158         args = ap.parse_args()
159         with open(args.manifest, encoding="utf-8") as f:
160             specs = json.load(f)
161         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
162
163
164     if __name__ == "__main__":
165         main()
166

```

12. assistant (2026-06-16T02:36:28.531Z)

Now I notice a critical discrepancy. The design claims `distill\_local.build\_candidates` produces a feature row matching the 5 fusion_golden shuttle features. Let me check the actual fusion_golden output schema and the CODE_MAP \$0 file-placement table, plus confirm the gt_loader strict behavior and `merge\_overlaps`.

13. user (2026-06-16T02:36:28.719Z)

```

1     """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3     Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4     candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5     EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6     writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7     CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8     measured by the shipped experiment harness; person features enter only as feature
columns the
9     learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):
12         python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16
17     """
18     from __future__ import annotations
19
20     import argparse
21     import json
22     import os
23     import time
24     from typing import Any, Dict, List, Optional
25
26     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

```

```

26     from backend.pipeline.detectors import rally_gate
27     from backend.pipeline.detectors import fusion_features as ff
28     from backend.eval import distill_local
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.training import label_candidates
31     from backend.utils.run_telemetry import RunTelemetry
32
33     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39     def _derive_video_path(spec: Dict[str, Any]) -> str:
40         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
41         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
42         convention)."""
43         if spec.get("video_path"):
44             return str(spec["video_path"])
45         labels = str(spec["labels"])
46         if labels.endswith(".rallies.csv"):
47             return labels[: -len(".rallies.csv")] + ".MP4"
48         raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
49         *.rallies.csv")
50
51     def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
52                      frame_skip: int = 3, iou: float = 0.5,
53                      log_every_sec: float = 15.0,
54                      telemetry: Optional[Any] = None) -> Dict[str, Any]:
55         """One golden video -> a feature record (shuttle + person features per candidate
56         window).
57         Returns `{name, fps, frame_width,
58         candidates:[{start,end,shuttle{ },person{ },label}], gts}`.
59         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
60         tests;
61         when None a real `PersonDetector` is built (loads torchvision on first use).
62         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
63         snapshots
64         from the detection progress callback so a hard kill leaves a forensic runlog.
65         """
66         fps, fw = float(spec["fps"]), float(spec["frame_width"])
67         traj = str(spec["trajectory"])
68         points = TrackNetRunner.parse_trajectory_csv(traj)
69         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
70         gts = load_golden_gts(str(spec["labels"]))
71         labels = label_candidates(intervals, gts, iou)
72         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
73         estimate
74         # (parity with the shuttle net-crossing gate) rather than hard-coding an
75         orientation.
76         axis, net_px, _span = rally_gate.estimate_play_axis(points)
77         video_path = _derive_video_path(spec)
78
79         if detector is None:
80             from backend.pipeline.detectors.person_detector import PersonDetector
81             detector = PersonDetector({"indexing": {"use_gpu": True}})
82
83         # Single-pass person detection over ALL candidate windows (open the video ONCE, read
84         # forward ? no per-window seek). Then compute features per window (pure, no GPU).
85         n = len(intervals)
86         win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
87         print(f"[fusion_golden] {spec['name']}: {n} windows ? person-detecting in one
88         pass...", flush=True)

```

```

82     t0 = time.monotonic()
83     state = {"last": t0}
84
85     def _progress(done: int, total: int) -> None:
86         if telemetry is not None:
87             telemetry.snapshot(done, total, phase=spec["name"])
88             now = time.monotonic()
89             if now - state["last"] >= log_every_sec or done >= total:
90                 el = now - t0
91                 rate = done / el if el > 0 else 0.0
92                 eta = (total - done) / rate if rate > 0 else 0.0
93                 pct = 100 * done // total if total else 0
94                 print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
95                         f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
96                 state["last"] = now
97
98     per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
99                                                    progress_cb=_progress)
100
101     candidates: List[Dict[str, Any]] = []
102     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
103         det = per_window[i]
104         det_fw = det.get("frame_width") or fw
105         det_fh = det.get("frame_height") or 0.0
106         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
107         # normalisation fusion_features uses (x/width, y/height).
108         if axis == "x":
109             net_norm = net_px / det_fw if det_fw > 0 else 0.5
110         else:
111             net_norm = net_px / det_fh if det_fh > 0 else 0.5
112         sf, ef = win_frames[i]
113         shuttle_in = [p for p in points if sf <= p.frame <= ef]
114         person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
115                                            court_polygon=None, net=(axis, net_norm))
116         candidates.append({
117             "start": float(s), "end": float(e),
118             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
119             "person": {k: float(v) for k, v in person.items()},
120             "label": int(labels[i]),
121         })
122     return {
123         "name": spec["name"], "fps": fps, "frame_width": fw,
124         "candidates": candidates,
125         "gts": [[float(a), float(b)] for a, b in gts],
126     }
127
128
129     def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
130            fresh: bool = False, smallest_first: bool = False) -> List[str]:
131         with open(manifest_path, encoding="utf-8") as f:
132             specs = json.load(f)
133         if smallest_first:
134             # Process small videos first for quick feedback (the slow full match lands
last);
135             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
136             specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
137                                                  if os.path.isfile(s.get("trajectory", ""))
else 0))
138         os.makedirs(out_dir, exist_ok=True)
139         # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
140         # rotating runlog of velocity + machine health for the post-mortem. Constructing it

```

```

must never
141     # break extraction, so guard the build and fall back to no telemetry on any error.
142     telem: Optional[RunTelemetry]
143     try:
144         telem = RunTelemetry(os.path.join(out_dir, "run_telemetry.jsonl"),
label="fusion_golden")
145     except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
146         telem = None
147     written: List[str] = []
148     for vi, spec in enumerate(specs, 1):
149         out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
150         # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
151         # run left off instead of re-detecting). --fresh forces a redo.
152         if not fresh and os.path.isfile(out):
153             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
154                 f"-> {out} (use --fresh to redo)", flush=True)
155             written.append(out)
156             continue
157         print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
158         t0 = time.monotonic()
159         if telem is not None:
160             telem.event("video_start", spec["name"])
161         try:
162             data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
163         except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
164             if telem is not None:
165                 telem.event("error", spec["name"], err=str(e))
166             raise
167         n_pos = sum(c["label"] for c in data["candidates"])
168         with open(out, "w", encoding="utf-8") as f:
169             json.dump(data, f, indent=2)
170         if telem is not None:
171             telem.event("video_done", spec["name"],
172                         windows=len(data["candidates"]), positives=n_pos)
173         print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
174             f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
175             f"-> {out}", flush=True)
176         written.append(out)
177     return written
178
179
180     def main() -> None:
181         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
182         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
183         ap.add_argument("--out-dir", default="output")
184         ap.add_argument("--frame-skip", type=int, default=3)
185         ap.add_argument("--iou", type=float, default=0.5)
186         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
187         ap.add_argument("--smallest-first", action="store_true",
188                         help="process smaller videos first for quick feedback (LOVO is
order-independent)")
189         args = ap.parse_args()
190         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
191             smallest_first=args.smallest_first)
192
193

```



```

52         return videos
53
54
55     def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                 threshold: float = 0.5) -> Dict[str, Any]:
57         shuttle_only = experiment.Variant(name="shuttle_only",
58         feature_names=list(SHUTTLE_FEATURE_NAMES),
59         threshold=threshold)
60         fusion = experiment.Variant(name="shuttle_person_fusion",
61         feature_names=list(SHUTTLE_FEATURE_NAMES) +
62         list(PERSON_FEATURE_NAMES),
63         threshold=threshold)
64         return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
65
66     def format_result(res: Dict[str, Any], iou: float) -> str:
67         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
68         ds = delta_stats(res)
69         st = ds["sign_test"]
70         lines = [
71             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
72             f" shuttle-only : F1={a['f1']:.3f} P={a['precision']:.3f} R={a['recall']:.3f}
73             f" IoU={a['mean_iou']:.3f}",
74             f" +person : F1={b['f1']:.3f} P={b['precision']:.3f} R={b['recall']:.3f}
75             f" IoU={b['mean_iou']:.3f}",
76             f" ?F1 = {d['f1']:+.3f} ({res['delta_pct']['f1']:+.1f}%)"
77             f"?P={d['precision']:+.3f} ?R={d['recall']:+.3f}",
78             f" paired ?F1 {ds['mean_delta']:+.3f} [95% CI {ds['ci_low']:+.3f},
79             f"{ds['ci_high']:+.3f}] "
80             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f}) "
81             f"| CI clears 0: {ds['ci_excludes_zero']}",
82             f" per-video ?F1:",
83         ]
84         for pv in res["per_video_delta"]:
85             lines.append(f" {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
86         return "\n".join(lines)
87
88     def main() -> None:
89         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
90         shuttle+person (golden set).")
91         ap.add_argument("--features-dir", default="output")
92         ap.add_argument("--iou", type=float, default=0.5)
93         ap.add_argument("--record", action="store_true", help="append the result to the
94         experiment leaderboard")
95         args = ap.parse_args()
96
97         videos = load_videos(args.features_dir)
98         if len(videos) < 2:
99             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
100             {len(videos)})")
101         res = compare(videos, iou=args.iou)
102         print(format_result(res, args.iou))
103         if args.record:
104             row = experiment.record_experiment(res)
105             print(f"recorded -> eval_baselines/experiment_leaderboard.json
106             ({row['timestamp']}")
107
108     if __name__ == "__main__":
109         main()

```

Now I have strong verification. A few important findings emerge. Let me check the CODE_MAP \$0 file-placement table, the gt_loader strict contract, and confirm whether `experiment.predict(CONTROL)` actually re-derives the windowing brain (it does NOT ? it operates on already-built intervals).

16. user (2026-06-16T02:36:34.602Z)

b6a39c4 feat(runner): fully-local no-AI segmenter mode (+ WASB stream flag, .env load) (#183)
---SECTION 0---

17. user (2026-06-16T02:36:36.470Z)

```
7:[Omitted long matching line]
32:| A test | `tests/test_*.py` | pytest auto-discovers; the full-suite lane has a **70%
coverage floor**. Pure/offline + mocked (no GPU/WSL/network). |
36:| Eval baseline / leaderboard JSON | `eval_baselines/` | Tracked (survives a clean checkout);
the no-regression gate reads it. |
37:| Pipeline run outputs (reels, trajectories, DBs, frames) | `output/` | **Gitignored.** Per-
video isolation is planned in `PER_VIDEO_OUTPUT_LAYOUT.md`. |
42:(grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic `setup.py`
moved to
66:     per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
114:[Omitted long matching line]
141:[Omitted long matching line]
142:[Omitted long matching line]
158:[Omitted long matching line]
174:[Omitted long matching line]
178:[Omitted long matching line]
179:[Omitted long matching line]
216:[Omitted long matching line]
290:- **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST
exist in `output/<db_name>` before any eval/regression.
327:| Run the whole test suite | `@run_all_tests.py` (repo root); or `pytest tests/` directly |
336:## 8. TEST MAP (`tests/test_*.py` -> subsystem)
```

18. user (2026-06-16T02:36:36.755Z)

```
1     """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3     The repo historically carried two incompatible loaders:
4     - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5       ``curate export`` format), raises on bad rows;
6     - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7       (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9     A file in one convention fed to the other loader yielded zero/garbage intervals with no
10    error. This module accepts BOTH conventions (header-mapped, else positional), so every
11    quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12    flows through one reader. Both legacy entry points now delegate here.
13
14    Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15    - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16      columns ? rally_number, ending_reason, sport, ? ? are ignored);
17    - no header: columns 0,1 positionally.
18    Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19    (``annotations.parse_timestamp``).
20    """
21    import csv
22    import logging
23    from typing import List, Optional, Tuple
24    from backend.eval.annotations import parse_timestamp
```

```

25
26     logger = logging.getLogger(__name__)
27
28     Interval = Tuple[float, float]
29
30     _START_NAMES = ("start", "start_time")
31     _END_NAMES = ("end", "end_time")
32
33
34     def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35         """If `row` is a header naming start/end columns, return their indices, else
None."""
36         cells = [(c or "").strip().lower() for c in row]
37         start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
38         end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
39         if start_idx is not None and end_idx is not None:
40             return start_idx, end_idx
41         return None
42
43
44     def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45         """Load GT rally intervals from either golden-CSV convention.
46
47         strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48         (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49         strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50         skipped ? a silently shrinking GT set flatters precision.
51
52         Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53         (they break one-to-one matching and usually indicate a labeling error).
54         """
55         intervals: List[Interval] = []
56         skipped = 0
57         cols: Optional[Tuple[int, int]] = None    # (start_idx, end_idx); None until detected
58         saw_header = False
59
60         with open(csv_path, newline="", encoding="utf-8-sig") as f:
61             for lineno, row in enumerate(csv.reader(f), start=1):
62                 if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                     continue
64                 if cols is None and not saw_header:
65                     header = _header_columns(row)
66                     if header is not None:
67                         cols = header
68                         saw_header = True
69                         continue          # consume the header row
70                 cols = (0, 1)           # headerless: positional start,end
71                 assert cols is not None  # every branch above assigns it
72                 si, ei = cols
73                 try:
74                     if len(row) <= max(si, ei):
75                         raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76                     start = parse_timestamp(row[si])
77                     end = parse_timestamp(row[ei])
78                     if start < 0:
79                         raise ValueError(f"negative start ({start})")
80                     if start >= end:
81                         raise ValueError(f"start ({start}) >= end ({end})")
82                 except ValueError as e:
83                     if strict:
84                         raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85                     skipped += 1
86                     continue
87                 intervals.append((start, end))

```

```

88
89         if skipped:
90             logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92         intervals.sort(key=lambda iv: iv[0])
93         for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94             if s2 < e1:
95                 logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                               "labeling error? Overlaps break one-to-one matching.",
97                               csv_path, s1, e1, s2, e2)
98         return intervals
99

```

19. assistant (2026-06-16T02:36:41.791Z)

I now have enough verification to identify a few real design discrepancies in addition to the requested gap analysis. Let me confirm two final things: whether ``load_golden_gts`` (used by ``fusion_golden`/`distill_local`/`served_gate_a``) is the lenient loader vs. the strict ``gt_loader`` the design specifies, and whether ``merge_overlaps`` exists where the design claims (``gemini_refine`` vs the design's claim of "merge_overlaps").

20. user (2026-06-16T02:36:45.400Z)

```

backend/eval/calibrate_wasb.py:33: def load_golden_gts(csv_path: str) -> List[Interval]:
---merge_overlaps def---
backend/eval/gemini_refine.py:39: def merge_overlaps(intervals: List[Interval], gap_tol: float =
0.0) -> List[Interval]:
---ablation litmus reproduces gate a tolerances---
backend/eval/splits.py:7:- assert_disjoint(train_ids, eval_ids) raises on any overlap.
backend/eval/splits.py:14:     assert_disjoint(train, eval_)
backend/eval/splits.py:74: def assert_disjoint(train_ids: Iterable[str], eval_ids: Iterable[str])
-> None:

```

21. user (2026-06-16T02:36:46.348Z)

```

33: def load_golden_gts(csv_path: str) -> List[Interval]:
38:     driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.

```

22. assistant (2026-06-16T02:36:55.050Z)

I have verified the key load-bearing claims. Two material discrepancies confirmed:

1. `**`load_golden_gts`` (used by ``fusion_golden`/`distill_local`/`fusion_compare``) is LENIENT (skips bad rows)\*\* ? but the design's `labels.csv` seam specifies ``load_gt_intervals(strict=True)``. The replay path through ``fusion_golden`/`distill_local`` does NOT use the strict loader. So a malformed committed label would NOT "fail loudly with file:line" on the quality-floor gate's actual load path ? the design conflates two different loaders.

2. `**`experiment.predict(CONTROL)`` operates on already-built ``intervals``, not the trajectory.\*\* The windowing brain (``trajectory_to_action_windows``) is upstream of ``experiment.predict``. So the characterization gate's claim that re-running ``experiment.predict(CONTROL)->merge_overlaps`` "characterizes the windowing brain" is only true if the gate ITSELF re-runs ``parse_trajectory_csv -> build_windows`` first (which the design does say in the `characterizationGate` section, but the ``expected_segments.json`` recompute via ``experiment.predict(CONTROL)`` does not re-derive windows ? it needs the explicit ``build_windows`` step the design mentions only in the characterization gate, not in the freeze tool's segment computation). This is internally inconsistent but recoverable.

I have enough to produce the gap analysis.

23. user (2026-06-16T02:38:40.110Z)

Output does not match required schema: root: must have required property 'criticalIssues', root:

must have required property 'missingPieces', root: must have required property 'falseSafetyRisks', root: must have required property 'verdict'

24. assistant (2026-06-16T02:38:43.599Z)

The tool wrapped my call incorrectly. Let me resubmit with the correct top-level structure.

25. user (2026-06-16T02:39:44.040Z)

Output does not match required schema: root: must have required property 'criticalIssues', root: must have required property 'missingPieces', root: must have required property 'falseSafetyRisks', root: must have required property 'verdict'

26. assistant (2026-06-16T02:39:47.800Z)

The harness is reporting the four properties as missing even though I'm providing them. This suggests the arguments are being nested under an unexpected wrapper key. Let me retry with a minimal payload to isolate whether it's a structural issue versus a content/size issue.

27. user (2026-06-16T02:39:48.757Z)

Structured output provided successfully

28. assistant (2026-06-16T02:39:52.053Z)

The minimal payload succeeded, so the structure is correct ? the earlier failures were a transient validation issue, likely payload size. Let me resubmit the full analysis, keeping it complete but tightened to stay within limits.

29. user (2026-06-16T02:40:38.602Z)

Output does not match required schema: root: must have required property 'criticalIssues', root: must have required property 'missingPieces', root: must have required property 'falseSafetyRisks', root: must have required property 'verdict'

30. assistant (2026-06-16T02:40:42.103Z)

The minimal payload passed but the full one fails ? this is a payload-size limit, not a structure problem. I'll trim the content substantially while preserving every finding's substance.

31. user (2026-06-16T02:41:20.231Z)

Output does not match required schema: root: must have required property 'criticalIssues', root: must have required property 'missingPieces', root: must have required property 'falseSafetyRisks', root: must have required property 'verdict'

32. assistant (2026-06-16T02:41:26.577Z)

Still failing at this size while the tiny payload passed. The cutoff is between them. Let me bisect: trim each field to roughly half to find the working envelope, prioritizing the two HIGH issues and the verdict.

33. user (2026-06-16T02:41:56.252Z)

Output does not match required schema: root: must have required property 'criticalIssues', root: must have required property 'missingPieces', root: must have required property 'falseSafetyRisks', root: must have required property 'verdict'

34. assistant (2026-06-16T02:42:00.909Z)

Still failing. The error names all four required properties as missing, which for a size issue is odd ? it usually points at the actual envelope. Let me test whether a SINGLE specific field

has a problematic character (the ``delta-F1``, ``<slug>``, ``>=``, or ``#146`` tokens), by submitting a medium payload that drops all special characters like ``<``, ``>``, ``#``, and ``::``.

35. user (2026-06-16T02:42:27.117Z)

Output does not match required schema: root: must have required property 'criticalIssues', root: must have required property 'missingPieces', root: must have required property 'falseSafetyRisks', root: must have required property 'verdict'

36. assistant (2026-06-16T02:42:32.132Z)

Removing special characters didn't help, so it is size, not content. The tiny payload passed. Let me find the threshold by submitting a moderately-sized version ? roughly 3 short critical issues, 4 short missing pieces, trimmed risks and verdict.

37. user (2026-06-16T02:42:52.401Z)

Output does not match required schema: root: must have required property 'criticalIssues', root: must have required property 'missingPieces', root: must have required property 'falseSafetyRisks', root: must have required property 'verdict'

38. assistant (2026-06-16T02:42:56.070Z)

Still failing at a moderate size. The threshold must be lower than I think, or the issue isn't pure character count but total token count of one field. Let me test the boundary precisely: keep arrays short (2-3 items each) with concise but complete content.

39. user (2026-06-16T02:43:11.482Z)

Output does not match required schema: root: must have required property 'criticalIssues', root: must have required property 'missingPieces', root: must have required property 'falseSafetyRisks', root: must have required property 'verdict'

40. assistant (2026-06-16T02:43:15.382Z)

Still failing. The boundary is lower than this. Let me go back near the known-good tiny size and grow carefully ? test with just 2 short critical issues, 2 missing pieces, 1 risk, short verdict.

41. user (2026-06-16T02:43:20.782Z)

Output does not match required schema: root: must have required property 'criticalIssues', root: must have required property 'missingPieces', root: must have required property 'falseSafetyRisks', root: must have required property 'verdict'

42. assistant (2026-06-16T02:43:25.884Z)

This is small and still failing, while my "test" payload passed. The difference must be content, not size. The one that passed used ``"test issue"`` / ``"test rec"`` etc. Let me check: maybe the validator rejects when a string contains a comma inside an array element, or the JSON in ``criticalIssues`` isn't parsing because of an embedded quote or apostrophe. My current payload has no apostrophes. Let me test the exact same shape as the passing one but with my real severity values and slightly longer text, to isolate.

43. user (2026-06-16T02:43:28.657Z)

Structured output provided successfully

44. assistant (2026-06-16T02:43:33.788Z)

That passed. The difference from my failing attempts: this one used `**separate XML parameter tags**` (``<parameter name="criticalIssues">...``) whereas all my failing attempts put everything inside a single concatenated block where I wrote the parameters as one run. The issue was that

in the large attempts, the parameter boundaries weren't being parsed correctly due to the volume. Now I know the format works ? let me resubmit the full detailed analysis using proper separate parameter tags and the complete content.

45. user (2026-06-16T02:44:24.989Z)

Structured output provided successfully

46. assistant (2026-06-16T02:44:35.393Z)

The structured output was accepted. The analysis is complete.

Key relevant files I verified during the review (all absolute):

- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\regression.py` ? confirms the exit-3/incommensurability guard contract and the absent `eval\_baselines/regression\_baseline.json`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\experiment.py` ? confirms `predict(CONTROL)` operates on already-built `intervals`, not the trajectory (HIGH issue #2)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill\_local.py` and `backend\eval\fusion\_golden.py` ? confirm the replay path loads labels via `calibrate\_wasb.load\_golden\_gts` (the lenient loader), not `gt\_loader.load\_gt\_intervals(strict=True)` (HIGH issue #1)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\gt\_loader.py` ? the strict loader the design *claims* but does not route through
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.github\workflows\ci.yml` ? confirms the four jobs (lint/typecheck/import-smoke/full-suite, `--cov-fail-under=70`) that are the substitute manual gate during the Actions outage
- `C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test\_served\_gate\_a\_regression.py` ? the skip-if-absent template the new gates mirror, and the eval-vs-serve `\_PROPOSAL=(0.005,1.0,0.5)` provenance the false-safety risk hinges on